



PREMIUM STUDY GUIDE

MASTER POWER BI IN 30 DAYS

Beginner to Pro — A Complete Day-by-Day Course
for Learners & Teachers

Beginner to Advanced | Power Query · Data Modeling · DAX · Visuals · Publishing
Includes worked DAX/M examples with real output, practice exercises, and 2 full projects

30

DAILY LESSONS

2

FULL PROJECTS

80+

DAX & M EXAMPLES

30

TEACHER'S NOTES

"A measure's result can change depending on the filters around it — understanding that is the whole game."

Table of Contents

WEEK 1 — Foundations (Days 1-7)

Day 1	Introduction to Power BI & the Interface
Day 2	Connecting to Data Sources
Day 3	Power Query Editor Basics
Day 4	Data Cleaning in Power Query
Day 5	Data Types & the Power Query M Language
Day 6	Merging & Appending Queries
Day 7	Data Modeling Basics: Relationships & Star Schema

WEEK 2 — DAX: Calculations (Days 8-14)

Day 8	Introduction to DAX: Columns vs Measures
Day 9	Basic DAX Functions
Day 10	The CALCULATE Function
Day 11	Time Intelligence
Day 12	Filter Functions: FILTER, ALL, ALLEXCEPT
Day 13	Logical & Text Functions
Day 14	Project 1: Sales Dashboard Data Model

WEEK 3 — Visuals & Report Design (Days 15-21)

Day 15	Introduction to Visualizations
Day 16	Advanced Visualizations
Day 17	Slicers & Filters
Day 18	Formatting & Themes
Day 19	Bookmarks & Drill-Through
Day 20	Tables & Matrix Visuals
Day 21	Week 3 Review: Concept Checkpoint

WEEK 4 — Publishing & Pro Practices (Days 22-30)

Day 22	Power BI Service: Publishing & Sharing
Day 23	Row-Level Security (RLS)
Day 24	Dataflows, Gateways & Incremental Refresh
Day 25	DAX Variables
Day 26	Iterator Functions: SUMX, AVERAGEX, RANKX
Day 27	What-If Parameters & Scenario Analysis
Day 28	Performance Optimization & Best Practices
Day 29	Report Design Best Practices
Day 30	Project 2 (Capstone): Executive Business Dashboard

WEEK 1

Power BI Foundations

Days 1–7: Understand what Power BI actually is, connect to real data sources, and shape and model that data correctly before building a single chart.

Why This Week Matters

Every impressive Power BI dashboard is built on top of unglamorous groundwork most viewers never see — clean data, correct types, and a well-built model of how tables relate. A shaky foundation here doesn't disappear as the course progresses; it resurfaces as confusing, subtly wrong numbers in Week 2's DAX and Week 3's visuals. Students who feel tempted to rush straight to building charts are exactly the ones who benefit most from slowing down here.

This Week's Lessons

DAY 1	Introduction to Power BI	Interface, Desktop vs Service vs Mobile
DAY 2	Connecting to Data Sources	Excel, CSV, Web, Databases
DAY 3	Power Query Editor Basics	Shaping data, Applied Steps
DAY 4	Data Cleaning	Duplicates, fill down, split columns
DAY 5	Data Types & M Language	Basic transformations
DAY 6	Merging & Appending Queries	Combining tables
DAY 7	Data Modeling Basics	Relationships, star schema + Week 1 project

By the End of This Week, You'll Be Able To

- Explain the difference between Power BI Desktop, Service, and Mobile
- Connect Power BI to Excel, CSV, and web data sources
- Clean and reshape data using the Power Query Editor
- Correctly set data types and understand basic Power Query M syntax
- Build a simple star-schema data model with correctly configured relationships

🔗 Teacher's Note

If you're teaching this week to a group, resist the urge to jump into flashy visuals early to keep engagement up. The habits formed in these first seven days — checking a column's data type before trusting a total, sketching a star schema before building relationships — are the same habits that prevent hours of confusing debugging once DAX and interactive reports enter the picture. Slow is smooth, and smooth is fast.

Introduction to Power BI & the Interface

🔗 Teacher's Note

Students almost always arrive expecting Power BI to be 'Excel with nicer charts,' and that assumption causes real confusion later. I always draw the three-part picture immediately: Power Query shapes data, the Data Model connects tables, and Reports visualize the result — three genuinely different jobs, not one big spreadsheet. Naming that separation on Day 1 saves enormous confusion once DAX and relationships enter the picture.

What Is Power BI?

Power BI is Microsoft's business intelligence platform — a set of tools for connecting to data, transforming it, modeling relationships between tables, and building interactive visual reports and dashboards. It's used by analysts, teams, and entire companies to turn raw data into decisions.

The Three Core Jobs Power BI Does

A common beginner mistake is treating Power BI like "Excel with nicer charts." It's actually three distinct tools working together:

Component	Job	Roughly equivalent to
Power Query	Connects to and CLEANS/SHAPES data	Preparing ingredients before cooking
Data Model	Defines how tables RELATE to each other	The recipe connecting ingredients
Reports/Visuals	DISPLAYS the data as interactive charts	Plating and serving the finished dish

Power BI Desktop vs Power BI Service vs Mobile

Product	What it's for	Runs on
Power BI Desktop	Building reports: connecting data, writing DAX, designing visuals	Windows application, free
Power BI Service	Publishing, sharing, and viewing reports online; scheduling refreshes	Web browser (app.powerbi.com)
Power BI Mobile	Viewing published reports on the go	iOS / Android app

You build in **Desktop**, then publish to the **Service** so others can view and interact with your report — even without Power BI installed themselves.

The Power BI Desktop Interface

Area	Purpose
Report View	Where you build and arrange visuals on report pages
Data View	Shows the actual rows of data in each loaded table
Model View	Shows tables and the relationships (lines) connecting them
Fields Pane	Lists all tables and columns available to drag onto visuals
Visualizations Pane	Lists chart types and formatting options for the selected visual

🔗 GETTING STARTED

1. Download **Power BI Desktop** for free from powerbi.microsoft.com
2. Open it — you'll land on the **Report View** by default, with a blank canvas
3. Notice the three view icons on the far left sidebar: Report, Data, and Model
4. Click each one once just to see what it shows — you'll return to all three constantly

🔗 **TIP:** If you're teaching a classroom, have every student open Power BI Desktop and just click through Report, Data, and Model view once before any real lesson content — a five-minute tour prevents a lot of 'where am I?' confusion later.

Why Companies Use Power BI

- Connects to nearly any data source: Excel, databases, cloud services, websites
- Automatically refreshes so reports stay current without manual rebuilding

- Interactive — viewers can filter, drill down, and explore, not just look at a static picture
- Scales from one person's personal report to an entire company's shared dashboards

🔗 **PRACTICE:** Install Power BI Desktop. Open it, and without loading any data yet, explore the Report, Data, and Model view icons, and identify the Fields pane and Visualizations pane on screen.

Quick Check

What's the key difference between Power BI Desktop and Power BI Service?

Answer: Desktop is where you BUILD reports (free Windows app); Service is where you PUBLISH, SHARE, and VIEW them online.

WEEK 1 — Foundations • DAY 2 OF 30

Connecting to Data Sources

🔗 Teacher's Note

Connecting to data feels like a trivial first step, but it's where students form their very first mental model of Power BI as a **TOOL THAT CONNECTS TO LIVE SOURCES**, not a place where you manually type data like a spreadsheet. I always ask: 'if the original Excel file changes tomorrow, what happens to your report?' — the answer (just hit Refresh) is the whole point of this lesson.

Power BI Connects to Live Data — It Doesn't Just Store It

Unlike typing data directly into a spreadsheet, Power BI is built to **connect** to a data source and pull data in. If the original source changes, you can simply **Refresh** instead of rebuilding anything by hand.

Common Data Sources

Source type	Examples	Common use case
Files	Excel, CSV, PDF, JSON	Departmental reports, exports from other systems
Databases	SQL Server, PostgreSQL, MySQL	Company transactional or operational data
Online services	SharePoint, Google Analytics, Salesforce	Cloud-based business tools
Web	Any public webpage with a data table	Quick one-off external data pulls

Connecting to an Excel File

🔗 IMPORT AN EXCEL WORKBOOK

1. Home tab → Get Data → Excel Workbook
2. Browse to and select your .xlsx file
3. In the Navigator window, check the box next to each sheet/table you want to load
4. Click **Transform Data** (recommended) to review and clean it first, or **Load** to bring it in directly

👁️ **WHAT YOU'LL SEE:** A Navigator window listing every sheet and named table in the workbook, each with a small preview of its first few rows — checked items will be loaded into your report.

Connecting to a CSV File

🔗 IMPORT A CSV FILE

1. Home tab → Get Data → Text/CSV
2. Browse to and select your .csv file
3. Power BI shows a preview and automatically detects column types and the delimiter
4. Click **Load** or **Transform Data**

Connecting to a Database

🔗 IMPORT FROM SQL SERVER (EXAMPLE)

1. Home tab → Get Data → SQL Server
2. Enter the server name and database name
3. Choose **Import** mode (loads a copy of the data into Power BI) or **DirectQuery** (queries the live database every time)
4. Select the tables you need in the Navigator, then Load or Transform

Import Mode vs DirectQuery — An Important Early Choice

	Import Mode	DirectQuery
Speed	Very fast — data lives inside the report file	Depends on the source database's speed
Data freshness	As current as your last Refresh	Always live, up-to-the-second
File size	Can get large with big datasets	Stays small — no data is copied
Best for	Most beginner-to-intermediate reports	Huge datasets, or data that must be real-time

⚠️ **WATCH OUT:** Beginners should default to Import mode unless a data source is genuinely too large to copy, or truly needs to be real-time. DirectQuery has real limitations on what DAX and transformations you can use.

⇒ **PRACTICE:** Download any small public CSV file (or use a sample dataset from Power BI's own sample files). Connect to it using Get Data → Text/CSV, and note what data types Power BI auto-detected for each column.

Quick Check

What's the main trade-off between Import mode and DirectQuery?

Answer: Import mode is faster and more flexible but only as current as your last refresh; DirectQuery is always live but depends on the source database's performance and has more limitations.

WEEK 1 — Foundations • DAY 3 OF 30

Power Query Editor Basics

🔗 Teacher's Note

The Power Query Editor is the single most under-appreciated screen in all of Power BI, and I spend real time here on purpose. Every single click a student makes gets recorded as a named, reorderable, deletable step — that's an enormously powerful and forgiving safety net, and most beginners don't realize it exists until they're shown the Applied Steps pane directly.

What Is the Power Query Editor?

The **Power Query Editor** is where you clean and reshape data BEFORE it loads into your report — removing columns you don't need, fixing data types, renaming things clearly, and more. Every action you take is recorded as a named, reorderable step.

🔗 OPENING THE POWER QUERY EDITOR

1. Home tab → Transform Data
2. A new window opens showing your data as a table, with a list of steps on the right

The Applied Steps Pane — Power Query's Safety Net

Every transformation you make — removing a column, renaming a field, changing a type — appears as a named step in the **Applied Steps** pane on the right. This is one of the most powerful, forgiving features in all of Power BI.

What you can do with Applied Steps	Why it matters
Click any step to see the data at that exact point	Understand exactly what each transformation did
Delete a step (click the X next to it)	Undo a specific change without redoing everything after it
Reorder steps (drag them)	Change the sequence transformations happen in
Rename a step	Document what a step does for your future self or teammates

Common Basic Transformations

Task	How
Remove a column	Right-click the column header → Remove
Keep only certain columns	Select the columns to keep → right-click → Remove Other Columns
Rename a column	Double-click the column header, type the new name
Reorder columns	Drag a column header to a new position
Change a column's data type	Click the data type icon in the column header, choose the correct type

Every Transformation Becomes M Code Behind the Scenes

Power Query records your clicks as code in a language called **M**. You rarely need to write M by hand as a beginner, but seeing it demystifies what's happening.

```

POWER QUERY M

let
    Source = Excel.Workbook(File.Contents("C:\Sales.xlsx"), null, true),
    Sheet1 = Source{[Item="Sheet1",Kind="Sheet"]} [Data],
    RemovedColumns = Table.RemoveColumns(Sheet1, {"InternalNotes"}),
    RenamedColumns = Table.RenameColumns(RemovedColumns, {"Amt", "Amount"})
in
    RenamedColumns

```

You can view this for your own query anytime via **View → Advanced Editor** — it's the exact, readable recipe of everything you clicked.

Loading Your Cleaned Data

FINISHING UP IN POWER QUERY

- Once your data looks correct, click **Home → Close & Apply**
- Power BI loads the cleaned data into your report and returns you to the Report View

TIP: Name your Applied Steps clearly as you go (right-click a step → Rename) — 'Removed unused columns' is far more useful to future-you than the generic default name Power Query gives it.

PRACTICE: Load any sample table into Power Query. Remove one column, rename another, and reorder two columns — then click through each Applied Step to see the data change at every stage.

Quick Check

What happens if you delete a step in the middle of the Applied Steps list?

Answer: Power Query removes just that transformation and tries to re-apply every step after it to the data as it existed before that step — sometimes this can cause errors if later steps depended on it.

WEEK 1 — Foundations • DAY 4 OF 30

Data Cleaning in Power Query

Teacher's Note

Data cleaning is where students first understand that a beautiful dashboard is built on top of unglamorous, tedious groundwork — and skipping it always shows up later as a visual that looks subtly wrong. I make students clean a genuinely messy sample file by hand once, so the temptation to skip Power Query and 'just build the report' gets addressed honestly before it becomes a habit.

Why Data Cleaning Matters

Real-world data is almost never ready to use immediately — duplicate rows, inconsistent spacing, dates split incorrectly across columns. A beautiful dashboard built on messy data still produces wrong or misleading numbers.

Removing Duplicate Rows

🔗 REMOVE DUPLICATES

1. Select the column(s) that should be unique
2. `Home tab → Remove Rows → Remove Duplicates`

⚠️ **WATCH OUT:** Remove Duplicates only compares the columns you've SELECTED, not the whole table — if you select just one column, rows will be considered duplicates based on that column alone, even if other columns differ.

Filling Down Blank Cells

A common pattern in exported reports: a category name is only listed once, with blanks below it for subsequent rows. **Fill Down** copies the last real value downward until the next actual value.

Before Fill Down

Region	Sales
North	5000
	7000
	4200
South	6000

After Fill Down on Region

Region	Sales
North	5000
North	7000
North	4200
South	6000

🔗 FILL DOWN

1. Right-click the column header
2. `Fill → Down`

Splitting a Column

Useful when one column actually contains two pieces of information, like a full name or a combined date-and-time.

🔗 SPLIT COLUMN BY DELIMITER

1. Right-click the column header
2. `Split Column → By Delimiter`
3. Choose the delimiter (comma, space, custom) and whether to split at the first, last, or every occurrence

Before splitting 'Full Name' by space

Full Name
Aisha Sharma
Ravi Kumar

After splitting

Full Name.1	Full Name.2
Aisha	Sharma
Ravi	Kumar

Trimming and Cleaning Text

Transformation	Fixes
Trim	Removes leading/trailing spaces
Clean	Removes non-printable characters
Capitalize Each Word / UPPERCASE / lowercase	Fixes inconsistent capitalization

🔗 ACCESS TEXT TRANSFORMATIONS

1. Select the text column
2. Transform tab → Format → (choose transformation)

Replacing Values

🔗 FIND AND REPLACE A VALUE

1. Right-click the column
2. Replace Values
3. Enter the value to find and what to replace it with

Useful for standardizing inconsistent entries, like fixing "USA", "U.S.A.", and "United States" into one consistent value.

💡 **TIP:** Clean data BEFORE building any visuals, not after — chasing a chart that 'looks wrong' back to a messy source column is far more frustrating than fixing the data first.

👉 **PRACTICE:** Take a small sample dataset with at least one duplicate row, one blank cell that should be filled down, and one column that should be split. Clean all three issues using the techniques above.

Quick Check

What does Fill Down do?

Answer: It copies the last non-blank value in a column downward into the following blank cells, until it reaches the next actual value.

WEEK 1 — Foundations • DAY 5 OF 30

Data Types & the Power Query M Language

🔗 Teacher's Note

Data types feel like a formality until a number quietly gets treated as text and every visual using it breaks in confusing ways. I always demonstrate this failure ON PURPOSE — import a column of numbers stored as text, try to SUM it, and let the class see the wrong result — because a demonstrated failure teaches the importance of data types far better than a rule stated in the abstract.

Why Data Types Matter So Much

A column's data type determines what Power BI is ALLOWED to do with it — sum a number, sort a date chronologically, or just treat it as plain text. Getting this wrong causes some of the most confusing beginner bugs in all of Power BI.

Common Power BI Data Types

Type	Icon meaning	Example
Whole Number	123	25, 1000, -3
Decimal Number	1.2	19.99, 3.14159
Text	ABC	'Aisha', 'North Region'
Date / Date-Time	calendar icon	2026-08-25
True/False	T/F	TRUE, FALSE

Changing a Column's Data Type

🔧 SET A COLUMN'S DATA TYPE

1. Click the small type icon on the left of the column header in Power Query
2. Choose the correct type from the dropdown
3. If prompted about replacing an existing step, choose Replace Current (usually simplest for a fresh change)

A Demonstration Worth Doing on Purpose

Sales column stored as TEXT (wrong)

Amount (Text)

500

750

300

RESULT

Sum of Amount

OUTPUT

Error or 0 — text can't be summed, even though it 'looks like' numbers

Sales column correctly stored as Decimal Number

Amount (Decimal)

500

750

300

RESULT

Sum of Amount

OUTPUT

1550

The values LOOK identical to a human eye — but only the correctly-typed column can actually be summed, averaged, or used in a numeric visual.

A Gentle Introduction to Power Query M

Every transformation is really M code behind the scenes. Here's a simple, complete M query read top to bottom:

```
POWER QUERY M

let
    Source = Csv.Document(File.Contents("C:\sales.csv")),
    PromotedHeaders = Table.PromoteHeaders(Source),
    ChangedTypes = Table.TransformColumnTypes(PromotedHeaders,
        {"Amount", type number}, {"OrderDate", type date})
in
    ChangedTypes
```

M concept

Meaning

let ... in

Defines a sequence of named steps, then returns the final one

Each line

One transformation, referencing the step before it by name

Table.TransformColumnTypes

The M function behind manually setting column types

Common Type-Related Errors

Error you might see	Likely cause
A visual shows 0 or blank for a number field	The column is stored as Text, not a number type
Dates sort alphabetically instead of chronologically	The column is Text instead of Date
"Error" appears in some cells after changing type	Some values in that column don't match the new type (e.g. a stray text value in a number column)

⚠ **WATCH OUT:** Always check data types immediately after connecting to a new source — Power BI's automatic detection is good but not perfect, especially with real-world messy exports.

🔗 **PRACTICE:** Load a small table with at least one numeric column. Deliberately set it to Text, try to build a Sum visual, observe the wrong/blank result, then fix the type back to a number and watch the visual correct itself.

Quick Check

Why might two columns that look identical on screen behave completely differently in a visual?

Answer: They may have different underlying data types — a number stored as Text looks the same as a real number to the eye, but can't be summed, averaged, or sorted numerically.

WEEK 1 — Foundations • DAY 6 OF 30

Merging & Appending Queries

🔗 Teacher's Note

Merging and appending are two of the most confused operations in Power Query, mostly because their everyday-English meanings overlap. I always use a physical metaphor: appending is stacking papers in the same pile (more rows); merging is stapling two different documents about the same person together side by side (more columns). That image sticks far better than the menu names alone.

Two Ways to Combine Tables

Power Query offers two fundamentally different ways to combine data from multiple tables — **Append** (stack rows) and **Merge** (join columns). Confusing the two is a very common beginner mistake.

Append — Stacking Rows on Top of Each Other

Use Append when you have the SAME columns across multiple tables and want to combine them into one longer table — like combining January, February, and March sales exports into one full year.

January_Sales

Date	Amount
2026-01-05	500
2026-01-12	300

February_Sales

Date	Amount
2026-02-03	700

🔗 APPEND QUERIES

1. Home tab → Append Queries
2. Choose whether to append two tables, or several at once (Append Queries as New)
3. Select the tables to stack together

Result: Combined_Sales (Appended)

Date	Amount
2026-01-05	500
2026-01-12	300
2026-02-03	700

Merge — Joining Columns Side by Side

Use Merge when you have RELATED tables with a shared key column and want to bring columns from one table into another — exactly like a SQL JOIN.

Orders

OrderID	CustomerID	Amount
1	101	500
2	102	300

Customers

CustomerID	Name
101	Aisha
102	Ravi

🔗 MERGE QUERIES

1. Home tab → Merge Queries
2. Select the two tables and click the matching column in each (e.g. CustomerID in both)
3. Choose a Join Kind (see table below)
4. Click OK, then expand the new merged column to choose which fields to bring in

Join Kinds — Power Query's Version of SQL JOINS

Join Kind	Equivalent to	Keeps
Left Outer	LEFT JOIN	All rows from the first table, matched where possible
Right Outer	RIGHT JOIN	All rows from the second table, matched where possible
Inner	INNER JOIN	Only rows that match in both tables
Full Outer	FULL OUTER JOIN	All rows from both tables
Anti (Left/Right)	NOT IN / NOT EXISTS	Only rows with NO match — great for finding orphaned data

Result: Orders merged with Customers (Left Outer)

OrderID	CustomerID	Amount	Name
1	101	500	Aisha
2	102	300	Ravi

Append vs Merge — Quick Decision Guide

Question to ask	If YES, use...
Do my tables have the SAME columns and I want more ROWS?	Append
Do my tables have DIFFERENT columns connected by a shared key, and I want more COLUMNS?	Merge

⚠ **WATCH OUT:** A very common beginner mistake: trying to Merge two tables that should have been Appended (or vice versa), producing a confusing result with far too many or far too few rows. Always ask 'do I want more rows or more columns?' before choosing.

⇒ **PRACTICE:** Create two small tables representing two months of sales with identical columns, and Append them. Then create an Orders table and a Products table connected by a ProductID, and Merge them using a Left Outer join.

Quick Check

If you want to add a Customer's name to an Orders table using a shared CustomerID, should you Append or Merge?

Answer: Merge — you're bringing in NEW COLUMNS from a related table based on a shared key, not stacking more rows with the same columns.

Data Modeling Basics: Relationships & Star Schema

🔗 Teacher's Note

This is the day students stop thinking about individual tables and start thinking about a **MODEL** — how tables relate to each other as a whole system. The star schema concept introduced here is genuinely one of the highest-leverage ideas in this entire course; a good model makes every later DAX formula easier, and a bad one makes even simple formulas fight you constantly.

From Individual Tables to a Data Model

So far, each lesson has focused on one table at a time. A **data model** is the full picture: multiple tables, and the **relationships** connecting them — this is what lets a single slicer filter visuals built from several different tables at once.

The Model View

🔗 VIEWING YOUR DATA MODEL

1. Click the **Model** icon on the far-left sidebar of Power BI Desktop
2. You'll see each loaded table as a box, with lines connecting related tables

What Is a Star Schema?

A **star schema** is the recommended way to structure a Power BI model — one or more central **fact tables** (transactional data, like Sales) surrounded by several **dimension tables** (descriptive data, like Products, Customers, and Dates) that each connect directly to the fact table.

```

      Products
      |
Customers ---- Sales (fact table) ---- Dates
      |
      Region

Sales contains: OrderID, ProductID, CustomerID, DateKey, RegionID, Amount, Quantity
Each dimension table connects to Sales via a shared key column

```

Table type	Contains	Example
Fact table	Transactions/events — usually many rows, mostly numbers	Sales (one row per order line)
Dimension table	Descriptive attributes — usually fewer rows, mostly text	Products, Customers, Dates, Region

Creating a Relationship

🔗 BUILD A RELATIONSHIP BETWEEN TWO TABLES

1. In Model view, drag from the key column in one table (e.g. Sales.ProductID)
2. Drop it onto the matching key column in the other table (Products.ProductID)
3. Power BI draws a connecting line and auto-detects the relationship type

🔍 WHAT YOU'LL SEE: A thin line connecting the two tables in Model view, with a '1' near the dimension table (Products) and a '*' (many) near the fact table (Sales), showing a one-to-many relationship.

Relationship Cardinality

Type	Meaning	Example
One-to-Many (1:*)	One row in table A relates to many rows in table B	One Product (1) appears in many Sales rows (*)
Many-to-Many (*:*)	Rows in both tables can relate to multiple rows in the other	Rare, generally avoided when possible
One-to-One (1:1)	Each row relates to exactly one row in the other table	Uncommon — often signals the tables should be merged instead

Filter Direction — Single vs Both

By default, filters flow from the "one" side (dimension) to the "many" side (fact) — selecting a Product filters the Sales table automatically. This default direction is correct for the vast majority of star-schema models.

⚠ **WATCH OUT:** Avoid 'Many-to-Many' relationships and bidirectional filtering unless you specifically understand why you need them — they can cause confusing, ambiguous filter behavior and slower performance. A clean star schema with one-directional, one-to-many relationships is the right default for beginners and most real reports.

💡 **TIP:** Sketch your star schema on paper BEFORE building it in Power BI — decide which table is your fact table and which are dimensions. This five-minute habit prevents a large share of relationship-related headaches later.

Week 1 Mini Project: Build a Simple Star Schema

Using a Sales fact table and Products/Customers/Dates dimension tables (sample data, or your own):

1. Load all four tables into Power BI via Get Data
2. Clean each table in Power Query — correct data types, remove unnecessary columns
3. In Model view, connect Sales to each dimension table using the shared key columns
4. Verify each relationship shows the correct 1-to-many direction

⇒ **PRACTICE:** Build the star schema described above using any 4 small sample tables you create yourself (even by typing a few rows into Excel first). Confirm in Model view that all relationships are one-to-many, flowing from each dimension table into Sales.

WEEK 2

DAX: The Language of Calculations

Days 8-14: Learn DAX from the ground up — calculated columns, measures, the crucial CALCULATE function, time intelligence, and filtering.

Why This Week Matters

This is the week Power BI stops just displaying data and starts calculating with it. DAX (Data Analysis Expressions) looks similar to Excel formulas, but it operates on an entirely different principle — context. A measure's result can change depending on which filters and visuals surround it, and understanding that single idea is the difference between fighting Power BI and using it fluently.

This Week's Lessons

DAY 8	Introduction to DAX	Calculated columns vs measures
DAY 9	Basic DAX Functions	SUM, AVERAGE, COUNT, aggregations
DAY 10	The CALCULATE Function	Changing filter context
DAY 11	Time Intelligence	TOTALYTD, SAMEPERIODLASTYEAR, DATEADD
DAY 12	Filter Functions	FILTER, ALL, ALLEXCEPT
DAY 13	Logical & Text Functions	IF, SWITCH, CONCATENATE
DAY 14	Project 1	Sales Dashboard Data Model

By the End of This Week, You'll Be Able To

- Explain the difference between a calculated column and a measure, and when to use each
- Write basic DAX aggregation formulas like SUM, AVERAGE, and COUNT
- Use CALCULATE to modify the filter context of a calculation
- Build common time intelligence measures like year-to-date and year-over-year growth
- Combine filter functions and logical functions to build real business measures

🔗 Teacher's Note

Watch for the moment a student starts predicting a measure's result correctly BEFORE running it — asking 'what filters are active right here?' before writing the formula. That predictive habit is the single biggest sign DAX's context model has genuinely clicked, far more than memorizing any individual function's syntax.

Introduction to DAX: Calculated Columns vs Measures

🔗 Teacher's Note

DAX is the topic most likely to make a confident Excel user feel like a total beginner again, and I address that head-on immediately: DAX LOOKS like Excel formulas, but the calculated-column-versus-measure distinction has no real Excel equivalent. I never let a student write their first measure without first explaining, out loud, exactly when it will and won't recalculate — that timing question is the real foundation everything else builds on.

What Is DAX?

DAX (Data Analysis Expressions) is the formula language used throughout Power BI to build calculations. It looks visually similar to Excel formulas — SUM(), IF(), and so on — but it works on entire tables and columns, and its results can change depending on filter context in a way Excel formulas don't.

Two Fundamentally Different Things You Can Build

	Calculated Column	Measure
Calculated when?	Once, when data refreshes	Every time it's used in a visual, recalculated live
Stored where?	As a new column, taking up memory in every row	Not stored as data — computed on demand
Responds to filters/slicers?	No — it's a fixed, pre-computed value per row	Yes — recalculates based on what's currently filtered
Typical use	Adding a fixed per-row attribute (e.g. a category label)	Aggregating and analyzing (totals, averages, ratios)

Creating a Calculated Column

🔗 ADD A CALCULATED COLUMN

1. In Data view or Model view, select the table
2. **Table tools** → **New Column**
3. Type the DAX formula in the formula bar

DAX

```
Profit = Sales[Revenue] - Sales[Cost]
```

RESULT

OrderID	Revenue	Cost	Profit
1	500	300	200
2	750	400	350

A new column added to every row of the Sales table

Creating a Measure

🔗 ADD A MEASURE

1. Select the table you want the measure logically grouped under
2. **Table tools** → **New Measure**
3. Type the DAX formula in the formula bar
4. Give it a clear, descriptive name before the equals sign

DAX

```
Total Revenue = SUM(Sales[Revenue])
```


RESULT	
	Total Revenue
(placed in a Card visual)	1,250
<i>Not stored per-row — recalculates instantly based on filters/slicers applied</i>	

Why This Distinction Matters So Much

Imagine a Total Revenue calculation. As a **calculated column**, it would show the SAME single number, repeated on every row, regardless of any filter or slicer applied. As a **measure**, it correctly shrinks or grows depending on what's currently filtered — by Region, by Date, by anything.

⚠ **WATCH OUT:** A very common beginner mistake: building a Total as a calculated column, then wondering why a slicer doesn't seem to affect a Card visual showing it. Aggregations that need to respond to filters almost always belong in a MEASURE, not a calculated column.

A Simple Rule of Thumb

Ask yourself...	Likely answer
Does this need a different value on every individual row?	Calculated column
Does this need to summarize/aggregate and respond to filters?	Measure
Am I building a total, average, count, ratio, or KPI?	Measure — almost always

💡 **TIP:** As a general rule for beginners: reach for a measure by default. Calculated columns are needed far less often than most new users initially assume — usually only for row-level labels or categories that Power Query couldn't easily produce.

⇒ **PRACTICE:** On a Sales table with Revenue and Cost columns, create a calculated column called Profit, and a separate measure called Total Profit using SUM. Place Total Profit in a Card visual and confirm it changes when you add a slicer and select different values.

Quick Check
 Why doesn't a calculated column respond to a slicer, while a measure does?
Answer: A calculated column is computed once, per row, when data refreshes — it's a fixed stored value. A measure recalculates on demand every time it's displayed, taking into account whatever filters are currently active.

WEEK 2 — DAX • DAY 9 OF 30

Basic DAX Functions: SUM, AVERAGE, COUNT

👨 **Teacher's Note**
 Basic DAX aggregation functions feel almost identical to their Excel counterparts, which is both reassuring and slightly dangerous — students sometimes assume DAX behaves EXACTLY like Excel and get surprised by context-dependent behavior later. I use this relatively gentle lesson to plant an early seed: 'a measure's result can change depending on WHERE you use it,' even though that idea won't fully click until CALCULATE arrives.

The Core DAX Aggregation Functions

Function	Purpose	Example
SUM()	Adds up all values in a column	SUM(Sales[Revenue])
AVERAGE()	Calculates the average of a column	AVERAGE(Sales[Revenue])
COUNT()	Counts numeric/date cells	COUNT(Sales[OrderID])
COUNTROWS()	Counts rows in a table	COUNTROWS(Sales)
MIN() / MAX()	Smallest / largest value	MAX(Sales[Revenue])
DISTINCTCOUNT()	Counts unique values	DISTINCTCOUNT(Sales[CustomerID])

Writing Your First Measures

DAX
 Total Revenue = SUM(Sales[Revenue])

RESULT	
Region	Total Revenue
North	45,000
South	38,000
East	52,000
Automatically breaks down by whatever's on the visual — here, Region	

DAX

```
Average Order Value = AVERAGE(Sales[Revenue])
```

RESULT	
Region	Average Order Value
North	1,125
South	950
East	1,300

DAX

```
Number of Orders = COUNTROWS(Sales)
```

RESULT	
Region	Number of Orders
North	40
South	40
East	40

COUNT vs COUNTROWS vs DISTINCTCOUNT

Function	Counts
COUNT(column)	Non-blank numeric or date values in that column
COUNTROWS(table)	Every row in a table, regardless of blanks
DISTINCTCOUNT(column)	Unique values only — great for 'how many different customers'

DAX

```
Unique Customers = DISTINCTCOUNT(Sales[CustomerID])
```

RESULT	
Region	Unique Customers
North	28
South	25
East	31

Implicit vs Explicit Measures

Dragging a numeric field directly onto a visual creates an **implicit measure** automatically (Power BI guesses SUM by default). Writing your own named measure (like Total Revenue above) is an **explicit measure** — always preferred for anything beyond a very quick, throwaway check.

	Implicit measure	Explicit measure
Created by	Dragging a field, letting Power BI auto-aggregate	Writing New Measure with DAX
Reusable across visuals with a clear name?	No	Yes
Can be customized with CALCULATE, filters, etc.?	No	Yes
Recommended for real reports?	Only for quick exploration	Yes — always for anything shared or reused

⚠ **WATCH OUT:** Relying only on implicit measures (just dragging raw fields) works fine for quick exploration, but makes reports harder to maintain and impossible to build advanced logic on top of. Build the habit of writing explicit measures early.

Naming Measures Clearly

Poor measure name	Better measure name
Sum of Revenue	Total Revenue
Measure 1	Year-to-Date Sales
Avg	Average Order Value

⇒ **PRACTICE:** On your Sales table, write four explicit measures: Total Revenue, Average Order Value, Number of Orders, and Unique Customers using DISTINCTCOUNT. Place all four in Card visuals and add a slicer by Region to confirm they respond correctly.

Quick Check

What's the difference between COUNT and COUNTROWS?

Answer: COUNT counts non-blank values within a specific COLUMN; COUNTROWS counts every row in a TABLE, regardless of whether individual columns have blanks.

WEEK 2 — DAX • DAY 10 OF 30

The CALCULATE Function

📌 Teacher's Note

CALCULATE is, without exaggeration, the most important function in this entire course — genuinely comparable in importance to JOIN in SQL or functions in Python. I never rush this lesson. The single reframing that helps most: CALCULATE doesn't just filter data, it changes the CONTEXT that every other DAX calculation around it operates inside. Everything from Day 11 onward assumes this has genuinely landed.

The Most Important Function in DAX

CALCULATE is, without exaggeration, the single most important function in DAX — comparable in importance to JOIN in SQL. It lets you take a measure and evaluate it under a MODIFIED filter context, rather than whatever filters happen to already be active.

DAX

```
CALCULATE(<expression>, <filter1>, <filter2>, ...)
```

A Motivating Example: North Region Sales, No Matter What

Imagine a report where users can slice by any region — but you also want ONE card that always shows North region sales specifically, regardless of what the user has selected elsewhere.

DAX

```
North Region Sales =
CALCULATE(
    SUM(Sales[Revenue]),
    Sales[Region] = "North"
)
```

RESULT	
Selected Slicer	North Region Sales (this measure)
(South selected)	45,000 — still shows North's total!
(East selected)	45,000 — unaffected by the slicer
CALCULATE's filter <i>OVERRIDES</i> the existing Region filter for this specific measure	

How CALCULATE Actually Works

CALCULATE takes an expression (usually another measure or aggregation) and one or more filter arguments. Those filters REPLACE any existing filter on the same column — they don't just add on top of it.

A Second Example: Comparing Against a Fixed Category

DAX	
<pre>High Value Orders = CALCULATE(COUNTROWS(Sales), Sales[Revenue] > 1000)</pre>	
RESULT	
Region	High Value Orders
North	12
South	8
East	15
Counts only orders above 1000, still broken down correctly by whatever's on the visual	

CALCULATE with Multiple Filters

DAX	
<pre>North Premium Sales = CALCULATE(SUM(Sales[Revenue]), Sales[Region] = "North", Sales[Category] = "Premium")</pre>	
RESULT	
	North Premium Sales
(any selection)	18,500
Multiple filter arguments are combined with AND logic	

CALCULATE and Context Transition

A subtler but important behavior: inside CALCULATE, any row-level filter you're currently inside (like a calculated column) gets converted into an equivalent filter context. This is called **context transition** — it's why CALCULATE is also essential inside calculated columns, not just measures.

The Mental Model That Makes CALCULATE Click

Without CALCULATE	With CALCULATE
A measure passively accepts whatever filters are already active on the report	A measure can actively OVERRIDE, ADD, or REMOVE specific filters
SUM(Sales[Revenue]) just totals whatever's currently visible	CALCULATE(SUM(Sales[Revenue]), condition) forces a specific slice, regardless of what's visible

 **TIP:** Whenever you think 'I need this number to ALWAYS show X, no matter what the user has filtered elsewhere' — that's the exact signal you need CALCULATE. It's the tool for deliberately overriding context.

⚠ **WATCH OUT:** CALCULATE filter arguments REPLACE existing filters on the same column, they don't simply narrow them further. Sales[Region]="North" inside CALCULATE will show North's data even if a slicer elsewhere is set to South — this surprises almost every beginner the first time.

⇒ **PRACTICE:** Write a measure called Premium Category Sales that always shows total revenue for the 'Premium' category using CALCULATE, regardless of what Category slicer selection is active elsewhere on the report. Test it by changing the slicer and confirming the measure doesn't change.

Quick Check

What does CALCULATE fundamentally let you do that a plain SUM() can't?

Answer: It lets you override, add, or remove specific filters for that one calculation — evaluating an expression under a deliberately modified filter context, rather than just whatever filters happen to already be active.

WEEK 2 — DAX • DAY 11 OF 30

Time Intelligence: YTD, SAMEPERIODLASTYEAR, DATEADD

🔗 Teacher's Note

Time intelligence functions are where DAX starts to feel like a superpower — a single line of code suddenly answers 'how does this compare to last year?' But that magic only makes sense once students realize these functions are really just specialized shortcuts built on top of CALCULATE and a date table, which is exactly why a clean, complete date table (from Week 1) is non-negotiable for this lesson to work correctly.

What Is Time Intelligence?

Time intelligence functions are specialized DAX shortcuts — built on top of CALCULATE — for common date-based comparisons: year-to-date totals, comparing to the same period last year, and more. They require a proper **Date table** in your model to work correctly.

Why You Need a Dedicated Date Table

Time intelligence functions rely on a continuous, unbroken calendar table marked as the model's official Date table — not just the date column buried inside your Sales table.

🔗 MARKING A TABLE AS THE DATE TABLE

1. Create or load a table with one row per calendar date, no gaps
2. Select the table → Table tools → Mark as Date Table
3. Choose the column containing the actual date values

DAX

```
CalendarTable =  
CALENDAR (DATE(2024,1,1), DATE(2026,12,31))
```

This DAX creates a complete, gap-free calendar between two dates — a common way to build a Date table from scratch.

TOTALYTD — Year-to-Date Totals

DAX

```
Revenue YTD =  
TOTALYTD(SUM(Sales[Revenue]), 'CalendarTable'[Date])
```

RESULT

Month	Revenue YTD
January	12,000
February	25,500
March	39,200
Running total from the start of the calendar year through each month	

SAMEPERIODLASTYEAR — Comparing to Last Year

DAX

```
Revenue LY =  
CALCULATE(  
    SUM(Sales[Revenue]),  
    SAMEPERIODLASTYEAR('CalendarTable'[Date])  
)
```

RESULT

Month	Revenue (This Year)	Revenue LY
March 2026	39,200	31,000
Same calendar period, one year earlier		

Calculating Year-over-Year Growth

DAX

```
YoY Growth % =  
DIVIDE(  
    [Total Revenue] - [Revenue LY],  
    [Revenue LY]  
)
```

RESULT

Month	YoY Growth %
March 2026	26.5%
A genuinely common, high-value business metric	

DATEADD — A More Flexible Shift

DAX

```
Revenue Prior Month =  
CALCULATE(  
    SUM(Sales[Revenue]),  
    DATEADD('CalendarTable'[Date], -1, MONTH)  
)
```

RESULT

Month	Revenue	Revenue Prior Month
March 2026	13,700	25,500
Shifts the date context back by any interval you specify		

Common Time Intelligence Functions

Function	Purpose
TOTALYTD / TOTALQTD / TOTALMTD	Running total from the start of the year/quarter/month
SAMEPERIODLASTYEAR	Same calendar period, one year earlier
DATEADD	Shift the date context by any number of days/months/quarters/years
DATESYTD	Returns the set of dates from year start — often used INSIDE CALCULATE

⚠ **WATCH OUT:** Time intelligence functions silently return incorrect or blank results without a proper, continuous Date table marked correctly in your model. If a time intelligence measure looks wrong, checking your Date table setup should be the very first thing you investigate.

💡 **TIP:** DIVIDE() is generally safer than the plain / operator in DAX — it automatically handles division-by-zero by returning blank (or a value you specify) instead of an error that can break a whole visual.

⇒ **PRACTICE:** Build a proper Date table for your Sales data using CALENDAR(), mark it as the official Date table, then write Revenue YTD and YoY Growth % measures using the patterns above.

Quick Check

Why do time intelligence functions require a dedicated Date table?

Answer: They rely on a continuous, gap-free calendar marked as the model's official date table to correctly calculate periods like 'year to date' or 'same period last year' — without one, the calculations can be wrong or fail entirely.

WEEK 2 — DAX • DAY 12 OF 30

Filter Functions: FILTER, ALL, ALLEXCEPT

🔑 Teacher's Note

FILTER and ALL are where students confront a subtlety that trips up even experienced Excel users: a filter and the ABSENCE of a filter are two completely different, equally important tools. I ask students to predict what ALL() does to a total before running it — the fact that it INCREASES a number, rather than restricting it like every other filter-sounding function they've met, is a genuinely counterintuitive moment worth sitting with.

FILTER — Building a Custom Filtered Table

FILTER returns a filtered version of a table, based on a condition — often used INSIDE CALCULATE for logic too complex for a simple column filter.

DAX

```
High Value Orders Revenue =  
CALCULATE(  
    SUM(Sales[Revenue]),  
    FILTER(Sales, Sales[Revenue] > 1000)  
)
```

RESULT

Region	High Value Orders Revenue
North	32,000
South	24,500
Only sums orders individually above 1000	

FILTER vs a Simple CALCULATE Condition

	Simple condition	FILTER
Syntax	CALCULATE(expr, Table[Column] = value)	CALCULATE(expr, FILTER(Table, condition))
When to use	Simple column comparisons	Complex conditions, multiple columns, or row-by-row logic
Performance	Generally faster	Can be slower on very large tables if overused

ALL — Removing Filters Entirely

ALL does the opposite of what its name suggests to newcomers — it REMOVES filters from a table or column, rather than restricting anything.

DAX

```
% of Total Revenue =  
DIVIDE(  
    SUM(Sales[Revenue]),  
    CALCULATE(SUM(Sales[Revenue]), ALL(Sales))  
)
```

RESULT

Region	Revenue	% of Total Revenue
North	45,000	32.1%
South	38,000	27.1%
East	57,000	40.8%
The denominator ignores the Region filter entirely, giving the grand total each time		

Why ALL Is Confusing (and How to Remember It)

Function	What it actually does
FILTER	Narrows a table down to matching rows — restricts
ALL	Removes existing filters from a table or column — expands back to everything

A helpful memory trick: **ALL** is asking for "all of the data, as if this filter never existed" — useful specifically for building percent-of-total calculations, since you need the unrestricted grand total as your denominator.

ALLEXCEPT — Removing Filters Selectively

ALLEXCEPT removes all filters from a table EXCEPT the ones you explicitly list — useful when you want to keep one grouping while ignoring everything else.

DAX

```
% of Region Total =  
DIVIDE(  
    SUM(Sales[Revenue]),  
    CALCULATE(SUM(Sales[Revenue]), ALLEXCEPT(Sales, Sales[Region]))  
)
```

RESULT

Region	Category	Revenue	% of Region Total
North	Premium	18,500	41.1%
North	Standard	26,500	58.9%

Denominator keeps the Region filter but ignores Category — giving each region's own total

Putting FILTER, ALL, and ALLEXCEPT Together

Function	Typical use case
FILTER	Complex row-level conditions inside CALCULATE
ALL(table)	Grand total ignoring every filter — percent of overall total
ALL(column)	Ignore just one specific column's filter
ALLEXCEPT(table, column)	Keep one specific filter, ignore everything else

💡 **TIP:** Percent-of-total calculations are one of the most common real-world uses of ALL — any time you see a report showing 'X% of total sales,' there's very likely an ALL() working quietly in the denominator.

⇒ **PRACTICE:** Using your Sales table, write a % of Total Revenue measure using ALL, then a % of Region Total measure using ALLEXCEPT — place both in a table visual broken down by Region and Category to see the difference.

Quick Check

What does ALL(Sales) do inside a CALCULATE?

Answer: It removes every existing filter on the Sales table for that calculation, effectively giving you the grand total as if no slicers or row context were applied at all.

WEEK 2 — DAX • DAY 13 OF 30

Logical & Text Functions: IF, SWITCH, CONCATENATE

👩 Teacher's Note

Logical and text DAX functions are the most immediately familiar territory for Excel users, and I use that comfort deliberately — this is a good day to rebuild confidence after CALCULATE and FILTER, before Week 2 ends with a genuinely challenging capstone project.

IF — Basic Conditional Logic

DAX

```
Sales Tier =  
IF(Sales[Revenue] >= 1000, "High Value", "Standard")
```

DAX

```
Sales Tier =  
IF(Sales[Revenue] >= 1000, "High Value", "Standard")
```

RESULT

OrderID	Revenue	Sales Tier
1	1500	High Value
2	600	Standard
A calculated column, applied per row		

Nested IF for Multiple Conditions

DAX

```
Sales Tier =  
IF(Sales[Revenue] >= 2000, "Platinum",  
    IF(Sales[Revenue] >= 1000, "Gold",  
        IF(Sales[Revenue] >= 500, "Silver", "Bronze")  
    )  
)
```

RESULT

OrderID	Revenue	Sales Tier
1	2500	Platinum
2	1200	Gold
3	700	Silver
4	200	Bronze

SWITCH — A Cleaner Alternative to Nested IF

DAX

```
Sales Tier =  
SWITCH(  
    TRUE(),  
    Sales[Revenue] >= 2000, "Platinum",  
    Sales[Revenue] >= 1000, "Gold",  
    Sales[Revenue] >= 500, "Silver",  
    "Bronze"  
)
```

RESULT

OrderID	Revenue	Sales Tier
1	2500	Platinum
2	1200	Gold
3	700	Silver
4	200	Bronze
Identical result to the nested IF above — but reads top to bottom far more clearly		

SWITCH(TRUE(), condition1, result1, condition2, result2, ..., default) checks each condition in order and returns the first match — the same logic as nested IF, written far more readably.

AND / OR — Combining Conditions

DAX

```
Is Priority Order =  
IF(  
    AND(Sales[Revenue] > 1000, Sales[Region] = "North"),  
    "Priority",  
    "Standard"  
)
```

DAX also supports the shorthand && for AND and || for OR, similar to many programming languages.

Common Text Functions

Function	Purpose	Example	Result
CONCATENATE(a,b)	Joins two text values	CONCATENATE("Order #", "1024")	Order #1024
a & b	Shorthand for joining text	"Order #" & 1024	Order #1024
UPPER() / LOWER()	Changes case	UPPER("hi")	HI
LEFT(text,n) / RIGHT(text,n)	First/last n characters	LEFT("Power BI",5)	Power
TRIM()	Removes extra spaces	TRIM(" hi ")	hi
LEN()	Length of text	LEN("Power BI")	8

A Practical Example: Building a Readable Label

DAX

```
Order Label =  
"Order #" & Sales[OrderID] & " (" & Sales[Region] & ")"
```

RESULT

OrderID	Region	Order Label
1024	North	Order #1024 (North)
1025	South	Order #1025 (South)

🔗 **TIP:** Reach for SWITCH(TRUE(), ...) instead of nested IF once you have more than two or three conditions — the readability difference becomes significant, and it's much easier to add a new tier later without breaking existing logic.

🔗 **PRACTICE:** Build a calculated column called Customer Segment using SWITCH(TRUE(), ...) with at least 3 tiers based on total customer spend, then build a text measure that combines a customer's name and segment into one readable label using &.

Quick Check

What does SWITCH(TRUE(), condition1, result1, condition2, result2, default) do?

Answer: It checks each condition top to bottom and returns the result for the first one that's TRUE, falling back to the default if none match — functionally identical to nested IF statements but far more readable.

WEEK 2 CAPSTONE • BEGINNER PROJECT

Project 1: Sales Dashboard Data Model

Project Overview

You're going to build a complete **Sales Dashboard Data Model** — connecting real tables, building a proper star schema, and writing a core set of business measures. This project uses **every skill from Week 1 and Week 2**: Power Query, relationships, and DAX.

The Schema

```

Products          Customers
+-----+          +-----+
| ProductID|      | CustomerID|
| Name      |      | Name      |
| Category  |      | Region   |
+-----+          +-----+

      |
      +----- Sales -----+
      | OrderID |
      | ProductID|
      | CustomerID|
      | Date     |
      | Revenue  |
      | Cost     |
      +-----+
      |
      CalendarTable
      +-----+
      | Date     |
      +-----+

```

Step 1: Load and Clean the Tables

🔧 POWER QUERY CLEANUP CHECKLIST

1. Connect to each of the 4 source tables (Products, Customers, Sales, and a generated CalendarTable)
2. Set correct data types on every column — especially Revenue/Cost as Decimal Number, Date as Date
3. Remove any unnecessary columns from each table
4. Close & Apply to load the cleaned tables into the model

DAX

```

CalendarTable =
CALENDAR (MIN(Sales[Date]), MAX(Sales[Date]))

```

Step 2: Build the Relationships

🔧 MODEL VIEW SETUP

1. Drag Products[ProductID] to Sales[ProductID]
2. Drag Customers[CustomerID] to Sales[CustomerID]
3. Drag CalendarTable[Date] to Sales[Date]
4. Mark CalendarTable as the official Date table
5. Confirm all three relationships show a 1-to-many direction from the dimension table into Sales

Step 3: Core DAX Measures

DAX

```

Total Revenue = SUM(Sales[Revenue])

Total Cost = SUM(Sales[Cost])

Total Profit = [Total Revenue] - [Total Cost]

Profit Margin % = DIVIDE([Total Profit], [Total Revenue])

Number of Orders = COUNTROWS(Sales)

Unique Customers = DISTINCTCOUNT(Sales[CustomerID])

```

Step 4: Add Time Intelligence

DAX

```
Revenue YTD =  
TOTALYTD([Total Revenue], 'CalendarTable'[Date])  
  
Revenue LY =  
CALCULATE([Total Revenue], SAMEPERIODLASTYEAR('CalendarTable'[Date]))  
  
YoY Growth % = DIVIDE([Total Revenue] - [Revenue LY], [Revenue LY])
```

Step 5: Add a Business Segmentation Measure

DAX

```
Customer Tier =  
SWITCH(  
    TRUE(),  
    [Total Revenue] >= 50000, "Platinum",  
    [Total Revenue] >= 20000, "Gold",  
    "Standard"  
)
```

RESULT

Customer	Total Revenue	Customer Tier
Aisha Retail Co.	62,000	Platinum
Ravi Traders	28,000	Gold
Meera Shop	9,500	Standard

Verifying Your Model

Check	How to confirm
Relationships are correct	Model view shows clean 1-to-many lines, no warning icons
Measures respond to filters	Add a Region slicer, confirm Total Revenue changes correctly
Time intelligence works	Revenue YTD increases month over month, Revenue LY shows a different, sensible number
No circular or ambiguous relationships	No error messages when loading the model

Extend the Project (Recommended)

- ☐ Add a Rank measure using RANKX to rank customers by Total Revenue
- ☐ Add a Top Product Category measure using CALCULATE + FILTER
- ☐ Add a Days Since Last Order measure using MAX and DATEDIFF
- ☐ Build one Card visual, one Table visual, and one Column chart using these measures to confirm they all work correctly (a preview of Week 3)

 **TIP:** This project mirrors almost exactly what a junior data analyst is asked to build in their first month: a clean model with a handful of well-named, correct core measures is worth far more than a flashy report built on a shaky foundation.

⇒ **PRACTICE:** Build the complete Sales Dashboard Data Model exactly as shown, verify all measures respond correctly to a test slicer, then implement at least one extension from the list above on your own.

WEEK 3

Visuals, Interactivity & Report Design

Days 15–21: Turn your data model into real, interactive reports — choosing the right visuals, adding slicers, formatting for clarity, and building drill-through navigation.

Why This Week Matters

This week marks a shift from calculating correct numbers to communicating them clearly. Week 2 was about getting the right value out of a DAX measure; Week 3 is about making sure a manager, client, or classmate can understand that number in seconds, and can explore further on their own. A report is where Power BI stops being a private calculation engine and starts being a genuine communication tool.

This Week's Lessons

DAY 15	Basic Visualizations	Bar, column, line, pie charts
DAY 16	Advanced Visualizations	Maps, treemaps, scatter, gauge
DAY 17	Slicers & Filters	Making reports interactive
DAY 18	Formatting & Themes	Conditional formatting, custom themes
DAY 19	Bookmarks & Drill-Through	Custom navigation
DAY 20	Tables & Matrix Visuals	Detailed, precise data views
DAY 21	Week 3 Review	Concept checkpoint

By the End of This Week, You'll Be Able To

- Choose the right chart type for a given business question
- Add slicers and filters to make a report genuinely interactive
- Apply conditional formatting and a consistent theme to a report
- Build a drill-through page and use bookmarks for guided navigation
- Decide when a table or matrix communicates better than a chart

Teacher's Note

This week tends to be the most fun to teach because the payoff is immediate and visual — a plain table of numbers becomes an interactive story in front of the class. Use that energy, but pair it with real restraint: this is also the week students most want to add every visual type at once. I always ask, 'what's the ONE question this page answers?' before letting a student add a fourth or fifth chart to it.

Introduction to Visualizations

🔗 Teacher's Note

Choosing a chart type is a decision students often make on autopilot — whatever Power BI suggests first, or whatever looks most impressive. I dedicate real time to the reasoning behind each choice rather than just the mechanics of dragging a field onto the canvas, because a chart is an argument, not a decoration, and the wrong chart type can accidentally misrepresent completely accurate data.

Adding Your First Visual

🔗 BUILD A COLUMN CHART

1. In Report view, click an empty area of the canvas
2. In the Visualizations pane, click the **Clustered Column Chart** icon
3. Drag **Region** from the Fields pane onto the **X-axis** well
4. Drag **Total Revenue** (your measure) onto the **Y-axis** well

👁️ WHAT YOU'LL SEE: A vertical bar chart appears, one bar per region, each bar's height showing that region's Total Revenue — automatically sorted and labeled.

Choosing the Right Chart Type

Chart Type	Best for	Example question it answers
Clustered Column / Bar	Comparing values across categories	Which region has the highest sales?
Line Chart	Showing a trend over time	How has revenue changed month to month?
Pie / Donut Chart	Showing parts of a whole (few categories only)	What share of sales comes from each category?
Card	A single, important number	What's our total revenue right now?
Table / Matrix	Precise, detailed values a user needs to read exactly	What's the exact revenue for every product?

Column Chart vs Bar Chart

These are the same chart, rotated 90 degrees. Use a **Bar Chart** (horizontal) when category names are long and would otherwise overlap on a column chart's x-axis.

Building a Line Chart for Trends

🔗 BUILD A LINE CHART

1. Click the **Line Chart** icon in the Visualizations pane
2. Drag **Date** (from your CalendarTable) onto the X-axis
3. Drag **Total Revenue** onto the Y-axis

👁️ WHAT YOU'LL SEE: A line rising and falling across months, letting a viewer instantly spot seasonal patterns or growth trends that a table of numbers would hide.

Building a Pie or Donut Chart

🔗 BUILD A DONUT CHART

1. Click the **Donut Chart** icon
2. Drag **Category** onto Legend
3. Drag **Total Revenue** onto Values

⚠️ WATCH OUT: Pie and donut charts become unreadable with more than about 5-6 slices, and are misleading if the values don't logically add up to a meaningful whole. When in doubt, a bar chart compares categories more clearly than a pie chart.

Building a Card Visual for a Single KPI

🔧 BUILD A CARD

1. Click the **Card** icon
2. Drag **Total Revenue** onto the Fields well

📊 **WHAT YOU'LL SEE:** A single large number displayed prominently — perfect for the one or two most important metrics at the top of a dashboard.

💡 **TIP:** Always add a clear title to every visual (Format pane → General → Title) — a chart without a title forces the viewer to guess what they're looking at, undermining even a perfectly correct chart.

👉 **PRACTICE:** Using your Sales Dashboard model from Project 1, build one Column chart (Revenue by Region), one Line chart (Revenue by Month), and two Cards (Total Revenue and Total Profit).

Quick Check

When should you avoid a pie chart?

Answer: When there are more than about 5-6 categories, or when the values don't add up to a meaningful whole — a bar chart almost always compares categories more clearly.

WEEK 3 — Visuals & Design • DAY 16 OF 30

Advanced Visualizations: Maps, Treemaps, Scatter, Gauge

👩‍🏫 Teacher's Note

Advanced visuals like maps and treemaps are exciting to students, but they're also the easiest way to accidentally build something that looks impressive and communicates almost nothing. I ask students to explain out loud, in one sentence, what question their chosen visual answers — if they can't do that quickly, it's usually the wrong visual for that particular data.

Scatter Chart — Showing Relationships Between Two Numbers

🔧 BUILD A SCATTER CHART

1. Click the **Scatter Chart** icon
2. Drag a numeric field (e.g. **Marketing Spend**) onto X-axis
3. Drag another numeric field (e.g. **Total Revenue**) onto Y-axis
4. Drag **Product** onto Details, so each dot represents one product

📊 **WHAT YOU'LL SEE:** A scatter of dots, one per product, letting a viewer visually spot whether higher marketing spend tends to correlate with higher revenue — a relationship a table of numbers wouldn't reveal at a glance.

Map Visuals — Showing Geographic Data

🔧 BUILD A MAP VISUAL

1. Click the **Map** (or Filled Map) icon
2. Drag a location field (e.g. **Country** or **City**) onto Location
3. Drag **Total Revenue** onto Size (for a bubble map) or Color saturation (for a filled map)

📊 **WHAT YOU'LL SEE:** A world or regional map with bubbles or shaded regions sized/colored by revenue — instantly showing WHERE your business is strongest, which a table of country names and numbers can't communicate as quickly.

⚠️ **WATCH OUT:** Map visuals need genuinely recognizable location data (country, state, city, postal code) to plot correctly — vague or misspelled location text often fails to map accurately. Clean location data in Power Query first.

Treemap — Hierarchical Part-to-Whole Comparisons

🔧 BUILD A TREEMAP

1. Click the **Treemap** icon
2. Drag **Category** onto Group, and **Sub-Category** underneath it for a second level
3. Drag **Total Revenue** onto Values

📊 **WHAT YOU'LL SEE:** Nested rectangles, sized proportionally to revenue — bigger rectangles for bigger categories, with sub-categories nested inside — useful for showing both totals AND hierarchy at once.

Gauge — Progress Toward a Target

🔧 BUILD A GAUGE VISUAL

1. Click the **Gauge** icon
2. Drag **Total Revenue** onto Value
3. Set a Target/Maximum value manually, or link it to a Target measure

📊 **WHAT YOU'LL SEE:** A speedometer-style dial showing how close current revenue is to a set target — commonly used for goal-tracking dashboards.

Choosing Among Advanced Visuals

Question you're answering	Best advanced visual
Do these two numbers relate to each other?	Scatter chart
Where geographically is this happening?	Map
How does a whole break down into nested categories?	Treemap
How close are we to a specific target?	Gauge

💡 **TIP:** Advanced visuals are more visually impressive, but also easier to misuse — before adding one, make sure you can state in one sentence exactly what question it answers. If you can't, a simpler chart is probably the better, clearer choice.

⇒ **PRACTICE:** Add a Scatter chart comparing two numeric fields in your dataset, and a Treemap breaking Total Revenue down by Category and Sub-Category (or Region, if Sub-Category isn't available).

Quick Check

What's a scatter chart best used for that a bar chart isn't?

Answer: Showing whether a *RELATIONSHIP* exists between two different numeric measures — like whether higher spend correlates with higher revenue — rather than just comparing totals across categories.

WEEK 3 — Visuals & Design • DAY 17 OF 30

Slicers & Filters: Making Reports Interactive

👤 Teacher's Note

Slicers are where a report stops being a static picture and starts being a genuine tool someone else can explore themselves — and that shift in what a 'report' even means is worth naming explicitly. This is usually the first moment non-technical students describe what they've built as 'an app' rather than 'a chart.'

From Static Charts to an Interactive Report

So far, every visual has shown ALL the data at once. **Slicers** and cross-filtering are what let a viewer explore the report themselves — clicking to filter everything on the page instantly, without needing you to build a separate chart for every possible view.

Adding a Slicer

🔧 BUILD A SLICER

1. Click the **Slicer** icon in the Visualizations pane
2. Drag **Region** onto the Field well
3. The slicer appears as a list of clickable region names on the canvas

📊 **WHAT YOU'LL SEE:** A small panel listing North, South, East, West as clickable buttons — clicking one instantly filters every other visual on the page that uses Region-related data.

Slicer Styles

Style	Best for
List	A small number of categories, need to select multiple easily
Dropdown	Saves space, good for many options
Between (for numbers/dates)	Selecting a numeric or date RANGE, like a date period
Relative Date	'Last 30 days', 'This Quarter' style quick date filtering

Cross-Filtering Between Visuals — Automatic Interactivity

By default, clicking a bar in ONE chart automatically filters every other visual on the page — no slicer required. Clicking "North" in a Region column chart will automatically filter a Product table alongside it.

📊 **WHAT YOU'LL SEE:** Clicking the 'North' bar in a column chart causes every other visual on the page to instantly highlight or filter down to North-only data, with the clicked bar itself staying solid while others fade slightly.

Controlling Cross-Filtering Behavior

🔧 ADJUST HOW VISUALS INTERACT

1. Click a visual
2. **Format tab → Edit Interactions**
3. Icons appear over every OTHER visual — choose Filter, Highlight, or None for how that visual reacts

Page-Level, Report-Level, and Visual-Level Filters

Filter type	Applies to
Visual-level filter	Just the one selected visual
Page-level filter	Every visual on the current report page
Report-level filter	Every page in the entire report

🔧 ADD A FILTER

1. Click the visual (for visual-level) or click empty canvas (for page-level)
2. Open the **Filters** pane on the right
3. Drag a field into the appropriate filter level, then set your condition

Slicer vs Filter — When to Use Each

	Slicer	Filters pane
Visible to report viewers?	Yes — an on-canvas visual	Only if you choose to show it
Best for	Interactive exploration you WANT viewers to control	Fixed rules you don't want casually changed

⚠️ **WATCH OUT:** Too many slicers on one page can overwhelm a viewer and clutter the design. A well-designed report page usually has 2-4 slicers at most — put more specific, situational filters in the Filters pane instead.

🔗 **PRACTICE:** Add a Region slicer and a Date range slicer to your Sales Dashboard report page. Click through different combinations and confirm every chart on the page updates correctly and consistently.

Quick Check

What's the difference between a page-level filter and a report-level filter?

Answer: A page-level filter only affects visuals on the current page; a report-level filter applies across every page in the entire report.

WEEK 3 — Visuals & Design • DAY 18 OF 30

Formatting & Themes

🔑 Teacher's Note

Formatting and conditional formatting are the lesson where technical skill and communication skill visibly merge, just like in a good spreadsheet dashboard. A technically correct report that nobody can read in ten seconds has failed at its actual job — I have students show a report to someone cold, with zero explanation, and report back what they understood.

Why Formatting Is Not Just Decoration

Formatting communicates meaning before a viewer even processes the underlying numbers — good formatting makes the right thing jump out immediately; poor formatting hides it in visual noise. Treat formatting decisions as "what does my viewer need to notice first?"

Conditional Formatting on a Table or Matrix

🔑 APPLY CONDITIONAL FORMATTING TO A COLUMN

1. Select your Table or Matrix visual
2. Click the field you want to format in the Values well → the small dropdown arrow
3. Conditional formatting → Background color (or Font color, Data bars, Icons)
4. Choose a rule: a color scale, rules based on value, or a formula

📊 WHAT YOU'LL SEE: A revenue column where high values automatically shade green and low values shade red, letting a viewer instantly spot underperforming rows without reading every number.

Conditional Formatting Types

Type	Effect
Background/Font color	Shades cells based on their value, like a heat map
Data bars	An in-cell bar proportional to the value, like Excel's data bars
Icons	Small arrows/traffic lights based on thresholds

Applying a Theme

🔑 APPLY A BUILT-IN OR CUSTOM THEME

1. View tab → Themes
2. Choose a built-in theme, or Browse for Themes to import a custom JSON theme file
3. Every visual's default colors update automatically to match

A consistent theme across an entire report — rather than default rainbow colors on every new visual — instantly makes a report look more professional and intentional.

Using Color with Purpose, Not Decoration

Do	Avoid
Use ONE consistent color to highlight the single most important series	A different random color for every category with no meaning
Use red/green sparingly, and consistently (e.g. always red = bad)	Using red for a HIGH sales number just because it looked available
Gray out less important context data	Making every visual equally bright and attention-grabbing

Formatting Numbers Clearly

🔧 SET A MEASURE'S DEFAULT FORMATTING

1. Select the measure in the Fields pane
2. `Measure tools` → `Format`
3. Choose Currency, Percentage, Whole Number, etc., and set decimal places

Raw value	Formatted for a business audience
1250000	\$1.25M or \$1,250,000
0.263	26.3%
3.14159265	3.14

Removing Visual Clutter

- Turn off unnecessary gridlines and borders (Format pane → Gridlines)
- Remove axis titles that repeat what's already obvious from context
- Keep a consistent font and size across all visuals on a page

💡 **TIP:** Ask a colleague to look at your report for five seconds and tell you the one thing they noticed — if it's not your main point, the formatting and layout likely need rethinking, not the underlying data.

⇒ **PRACTICE:** Apply conditional formatting (data bars or color scale) to a revenue column in a table visual, apply a consistent built-in theme to your whole report, and format every currency measure to show no more than 2 decimal places.

Quick Check

Why might using bright red for a GOOD result and green for a BAD one confuse a viewer?

Answer: Because red and green carry strong, near-universal conventional meanings (red = bad/stop, green = good/go) in business reporting — reversing them fights the viewer's instinctive expectations and slows down understanding.

WEEK 3 — Visuals & Design • DAY 19 OF 30

Bookmarks & Drill-Through

👤 Teacher's Note

Bookmarks and drill-through are genuinely some of the more advanced interactivity concepts in this course, and I introduce them only after students have built and used a few static reports themselves — the frustration of 'I wish clicking this could take me somewhere else' needs to be felt first for the feature to feel like a solution rather than an arbitrary trick.

What Is Drill-Through?

Drill-through lets a viewer right-click a data point on one page and jump to a DIFFERENT, more detailed page filtered specifically to that item — like clicking "North" on a summary page and landing on a page showing only North's detailed orders.

🔧 SET UP A DRILL-THROUGH PAGE

1. Create a new report page (e.g. 'Region Details')
2. In the Fields pane, drag **Region** into the **Drill through** well in the Visualizations pane
3. Build detailed visuals on this page as normal — they'll automatically respect the drill-through filter
4. A 'Back' button appears automatically in the top-left of the page

👁️ **WHAT YOU'LL SEE:** Right-clicking the 'North' bar on a summary page shows a context menu with 'Drill through → Region Details' — clicking it navigates to the details page, already filtered to North only, with a back arrow to return.

What Is a Bookmark?

A **bookmark** saves the current state of a report page — which filters are applied, which visuals are visible, even the current zoom/scroll position — so you can jump back to that exact view with one click, often used for guided walkthroughs or toggling between views.

🔖 CREATE A BOOKMARK

1. View tab → Bookmarks pane → Add
2. Set up the report exactly how you want it captured (filters applied, visuals shown/hidden)
3. Click Add — the bookmark saves that exact state with a name you can edit

A Practical Bookmark Use Case: Toggle Between Views

🔖 BUILD A SHOW/HIDE TOGGLE WITH BOOKMARKS AND BUTTONS

1. Build two versions of a visual (e.g. a Chart view and a Table view) on top of each other
2. Create a bookmark for each state, making sure to check 'Data' and 'Display' but hide the OTHER visual in each bookmark
3. Add two buttons (Insert → Buttons), and assign each button's action to jump to the matching bookmark

📌 **WHAT YOU'LL SEE:** Two small buttons labeled 'Chart View' and 'Table View' sitting above the same space on the page — clicking one instantly swaps which visual is visible, without navigating to a different page at all.

Drill-Through vs Bookmarks — When to Use Each

Need	Use
Jump to a MORE DETAILED page about one specific item	Drill-through
Save and restore a specific combination of filters/visibility	Bookmark
Build a guided, presentation-style walkthrough of a report	A sequence of bookmarks + a Bookmark Navigator

Built-In Drill-Down (Different from Drill-Through)

Don't confuse drill-through with the small drill-down arrows that appear automatically on hierarchical visuals (like Category → Sub-Category) — that's a simpler, built-in way to expand one level deeper WITHOUT leaving the current page.

⚠️ **WATCH OUT:** Bookmarks capture visual visibility and filters — but can behave unexpectedly if you significantly redesign a page's layout afterward. Re-check bookmarks after any major report redesign.

💡 **TIP:** Drill-through pages are one of the most professional-feeling features you can add to a report relatively quickly — they let one clean summary page lead into unlimited depth, without cluttering the main view.

➡ **PRACTICE:** Build a drill-through page from your Region summary chart showing detailed order-level data for whichever region is clicked. Then create two bookmarks toggling between a Chart view and a Table view of the same data, with buttons to switch between them.

Quick Check

What's the key difference between drill-through and a bookmark?

Answer: Drill-through navigates to a DIFFERENT page, automatically filtered to whatever was clicked; a bookmark saves and restores a specific state (filters, visibility) that you define manually ahead of time.

WEEK 3 — Visuals & Design • DAY 20 OF 30

Tables & Matrix Visuals

👩‍🏫 Teacher's Note

Tables and matrix visuals feel like a step backward after building flashy charts, but I remind students: sometimes the audience genuinely just wants the exact numbers, not a picture of the numbers. Knowing when NOT to use a chart is just as much a design skill as knowing which chart to pick.

When a Chart Isn't the Right Answer

Charts are great for spotting patterns and trends — but sometimes a viewer genuinely needs the EXACT numbers, not a picture of them. Knowing when NOT to use a chart is just as much a design skill as picking the right chart type.

The Table Visual

BUILD A TABLE

1. Click the **Table** icon in the Visualizations pane
2. Drag several fields (e.g. Product, Category, Revenue, Profit) onto the Columns well

 **WHAT YOU'LL SEE:** A simple grid of rows and columns, similar to a spreadsheet — every exact value visible and readable, sortable by clicking any column header.

The Matrix Visual — A Pivot Table Inside Power BI

BUILD A MATRIX

1. Click the **Matrix** icon
2. Drag **Region** onto Rows
3. Drag **Category** onto Columns
4. Drag **Total Revenue** onto Values

 **WHAT YOU'LL SEE:** A grid with Regions down the left side, Categories across the top, and Total Revenue filling each intersecting cell — exactly like a Pivot Table in Excel, but interactive and connected to the live model.

Table vs Matrix

	Table	Matrix
Layout	Flat rows and columns	Rows AND columns can both group/pivot data
Supports drill-down hierarchies?	No	Yes
Best for	A simple detailed list	Cross-tabulated summaries, like Region x Category

Adding Subtotals and Totals to a Matrix

ENABLE SUBTOTALS

1. Select the Matrix visual
2. **Format pane** → Subtotals
3. Toggle Row subtotals and Column subtotals on or off as needed

Sorting a Table or Matrix

Click any column header once to sort ascending, click again for descending — or use the "..." menu on the visual for a Sort By option, including sorting by a different (hidden) column.

Conditional Formatting Still Applies

Everything from Day 18's conditional formatting lesson (data bars, color scales, icons) works directly inside Table and Matrix visuals — often the single most useful place to apply it, since the exact numbers are already visible.

When to Choose a Table/Matrix Over a Chart

Situation	Better choice
Finance/accounting audience needing exact figures	Table or Matrix
Executive summary meant to be scanned in seconds	Chart
Comparing two dimensions at once (Region x Category)	Matrix
A detailed export-style page a user might screenshot or print	Table

 **TIP:** A well-designed report often mixes BOTH — Card and Chart visuals up top for a fast overview, with a detailed Table or Matrix further down (or on a drill-through page) for anyone who needs to dig into exact numbers.

 **PRACTICE:** Build a Matrix visual with Region on Rows, Category on Columns, and Total Revenue in Values, with subtotals enabled. Apply data bar conditional formatting to the Values, and compare how it reads versus the same data in a Column chart.

Quick Check

When would a Matrix visual be a better choice than a bar chart?

Answer: When you need to compare data across TWO dimensions at once (like Region and Category together) and the viewer needs exact numbers, not just a visual comparison — a matrix pivots data the way a chart can't.

WEEK 3 — Visuals & Design • DAY 21 OF 30

Week 3 Review: Concept Checkpoint

🔑 Teacher's Note

By the end of Week 3, students can both model data and present it well. What I watch for in the review isn't whether every visual type is memorized — it's whether a student can look at a new business question and correctly decide: does this need a KPI card, a trend line, or a detailed table? That judgment call is the real skill this week was building toward.

Week 3 Recap

Day	Skill
15	Column, line, pie/donut, and card visuals
16	Scatter, map, treemap, and gauge visuals
17	Slicers, cross-filtering, and filter levels
18	Conditional formatting and consistent themes
19	Drill-through pages and bookmarks
20	Table and Matrix visuals for detailed data

Concept Checkpoint: Choosing the Right Visual

For each business question below, decide which visual type fits best BEFORE checking the suggested answer — this is the real skill Week 3 was building.

Business question	A strong visual choice
What's our total revenue this month?	Card
How has revenue trended over the past 12 months?	Line chart
Which of our 6 product categories sells best?	Bar or column chart
Where geographically are our customers located?	Map
What's the exact revenue and profit for every single product?	Table
How does revenue break down by Region AND Category together?	Matrix
Does higher ad spend correlate with higher revenue?	Scatter chart

Building a Cohesive Report Page

A well-designed page isn't just a collection of correct visuals — it has a clear visual hierarchy and flow.

1. **Top:** 2-4 Cards for the most important KPIs, glanceable in seconds
2. **Middle:** 1-2 primary charts answering the page's main question
3. **Side or bottom:** Slicers for interactivity, and/or a detailed table for anyone who needs exact numbers
4. **Consistent theme** and formatting throughout, with restrained, purposeful use of color

A Simple Self-Review Checklist

- ☐ Does every visual have a clear title?
- ☐ Could a first-time viewer understand the main point of this page in 10 seconds?
- ☐ Is color used consistently and purposefully, not decoratively?
- ☐ Are slicers/filters easy to find and understand?
- ☐ Would a finance/detail-oriented viewer be able to find exact numbers somewhere on or near this page?

 **TIP:** If you're teaching this week's review in a classroom, have students swap completed report pages with a partner and give five seconds of silent viewing before asking what they understood — it's a fast, honest test of real report design, not just correct visual selection.

⇒ **PRACTICE:** Take your Sales Dashboard from Project 1 and build one complete, polished report page combining Cards, a primary chart, a slicer, and a detailed Table or Matrix — then run it through the self-review checklist above.

WEEK 4

Publishing, Advanced DAX & Pro Practices

Days 22–30: Publish and secure reports in the Power BI Service, master advanced DAX (variables, iterators), and design a complete capstone dashboard.

Why This Week Matters

The final week is less about new basic syntax and more about the tools that separate someone who can build a report from someone who can ship a secure, performant, genuinely professional one. Publishing and Row-Level Security connect your work to real people safely. Advanced DAX makes complex business logic both correct and readable. This is also the least hand-held week — real dashboard requests never arrive with a lesson number attached to them.

This Week's Lessons

DAY 22	Power BI Service	Publishing, workspaces, sharing
DAY 23	Row-Level Security	Restricting data by user
DAY 24	Dataflows & Incremental Refresh	Enterprise-scale data management
DAY 25	DAX Variables	VAR for readable, efficient formulas
DAY 26	Iterator Functions	SUMX, AVERAGEX, RANKX
DAY 27	What-If Parameters	Interactive scenario analysis
DAY 28	Performance Optimization	Star schema, DAX best practices
DAY 29	Report Design Best Practices	Professional dashboard design
DAY 30	Project 2	Capstone: Executive Business Dashboard

By the End of This Week, You'll Be Able To

- Publish a report to the Power BI Service and share it securely with others
- Restrict data access by user using Row-Level Security
- Write cleaner, more efficient DAX using variables
- Use iterator functions to perform row-by-row calculations across a table
- Design and build a complete, polished, multi-page executive dashboard

Teacher's Note

By this point, most of the remaining struggle isn't about understanding any single feature — it's about deciding which tool to reach for first when a real, messy dashboard request shows up with no instructions attached. That's exactly what the Day 30 capstone project is designed to test, and it's the truest measure of whether the last four weeks actually stuck.

Power BI Service: Publishing, Workspaces & Sharing

Teacher's Note

Publishing to the Power BI Service is the first moment a report becomes something OTHER people can actually use without needing Power BI Desktop installed on their own machine — a genuinely significant milestone that's easy to undersell. I make a point of having students share a report with a classmate and watch them interact with it live, since that's the real payoff of everything built so far.

From Desktop File to Shared Report

A .pbix file on your own computer is only useful to you. The **Power BI Service** (app.powerbi.com) is where reports get published, refreshed on a schedule, and shared with colleagues — even ones without Power BI Desktop installed.

Publishing a Report

PUBLISH FROM DESKTOP TO THE SERVICE

1. Home tab → Publish 
2. Sign in with a Power BI account (a free or organizational account)
3. Choose a **Workspace** to publish into (or use 'My Workspace' for personal reports)
4. Once published, click the link provided to open it directly in your browser

What Is a Workspace?

A **workspace** is a shared container in the Power BI Service where a team's reports, dashboards, and datasets live together — with controlled access for who can view versus edit.

Workspace type	Use case
My Workspace	Personal reports, just for you, not shared by default
Team/App Workspace	Reports meant to be built and shared collaboratively with a team

Workspace Roles

Role	Can do
Admin	Full control: manage workspace settings, add/remove members
Member	Publish and edit content within the workspace
Contributor	Edit content, but with slightly fewer permissions than Member
Viewer	View reports only, no editing

Sharing a Report

SHARE A PUBLISHED REPORT

1. Open the report in the Power BI Service
2. Share button (top right) 
3. Enter the email addresses of people to share with, and set their permission level
4. They'll receive a link (and need at least a free Power BI account to view it)

Scheduling a Data Refresh

A published report doesn't automatically pull new data forever — you need to configure how often it refreshes from the original source.

🔧 SET UP SCHEDULED REFRESH

1. In the workspace, find your dataset (not the report itself)
2. Settings (the "... menu) → Scheduled refresh
3. Toggle it on, choose a frequency (e.g. once daily), and set the time
4. For on-premises data sources, you'll also need a **Gateway** installed — see Day 24

Apps — A Polished Way to Distribute Reports

A Power BI **App** bundles one or more reports from a workspace into a clean, curated, read-only experience for a wider audience — often used when you want a polished front door rather than exposing the whole workspace directly.

⚠️ **WATCH OUT:** Sharing individual reports directly is fine for a small team, but as the audience grows, publishing a proper App (with defined navigation and a curated set of reports) gives a far more professional, controlled experience than ad-hoc sharing links.

💡 **TIP:** Always test a newly shared report by viewing it as a DIFFERENT user (or asking a colleague to open the shared link) rather than assuming your own view — permissions and Row-Level Security (tomorrow's lesson) can behave differently for other people.

⇒ **PRACTICE:** Publish one of your practice reports to a workspace in the Power BI Service, share it with a classmate or colleague's email, and set up a scheduled refresh for its dataset.

Quick Check

What's the difference between 'My Workspace' and a Team Workspace?

Answer: My Workspace is personal and not shared by default; a Team Workspace is a shared container meant for a group to collaboratively build, manage, and share reports together.

WEEK 4 — Pro Practices • DAY 23 OF 30

Row-Level Security (RLS)

👨‍🏫 Teacher's Note

Row-Level Security is a topic that deserves real caution alongside the technical steps — a correctly working RLS setup that LOOKS correct in testing can still leak data if a single edge case is missed. I always tell students: test RLS by actually viewing the report AS each restricted role, never just by reading the rule and assuming it's right.

The Problem Row-Level Security Solves

Imagine a Sales report shared company-wide — but each regional manager should only see THEIR OWN region's data, not everyone else's. **Row-Level Security** (RLS) restricts what data each user sees, using the SAME published report for everyone.

Creating a Security Role

🔧 DEFINE AN RLS ROLE IN POWER BI DESKTOP

1. Modeling tab → Manage Roles
2. Click **Create** and name the role (e.g. 'North Region Manager')
3. Select the table to filter (e.g. Sales), and write a DAX filter expression

DAX

```
[Region] = "North"
```

This filter expression determines exactly which rows a user assigned to this role will be able to see — everywhere in the report, across every visual.

Testing a Role Before Publishing

🔧 VIEW AS A SPECIFIC ROLE

1. Modeling tab → View As
2. Check the role you want to test (e.g. 'North Region Manager')
3. The report canvas updates live to show exactly what that role would see

⚠ **WATCH OUT:** ALWAYS test RLS using View As before publishing — a rule that looks correct on paper can still have mistakes that only become obvious when you actually see the restricted view in action.

A More Dynamic Approach: User-Based RLS

Instead of one role per region, a more scalable pattern uses the logged-in user's email to dynamically filter data — especially useful when you have many regions or salespeople.

DAX

```
[SalesRepEmail] = USERPRINCIPALNAME()
```

USERPRINCIPALNAME() returns the email address of whoever is currently viewing the report — combined with a mapping table (SalesRep → Email → Region), this scales to any number of users without creating a role per person.

Assigning Users to Roles in the Service

🔑 ASSIGN ROLES AFTER PUBLISHING

1. In the Power BI Service, go to the dataset's settings
2. **Security**
3. Select a role, then add the specific users or security groups who should have it

Static Roles vs Dynamic RLS

	Static role (one per region)	Dynamic RLS (USERPRINCIPALNAME)
Setup effort	Simple for a few fixed categories	More setup, but scales far better
Best for	A small, fixed number of groups	Many users, or frequently changing assignments
Maintenance	Manually add each user to a role	Automatically driven by a mapping table

RLS Only Restricts Viewing, Not Report Design

RLS applies in the Power BI Service to report viewers — anyone with edit access to the underlying file in Desktop can still see everything unless additional workspace-level permissions are also configured.

💡 **TIP:** Think of RLS filters the same way you'd think of a WHERE clause quietly applied to every single query behind every visual — once defined and assigned correctly, the user never has to do anything themselves; the restriction is completely automatic and invisible to them.

🔑 **PRACTICE:** Create a static RLS role restricting Sales data to one specific Region using a DAX filter, test it thoroughly with View As, then research (even without implementing) how USERPRINCIPALNAME() could make this dynamic for many regions at once.

Quick Check

Why is testing RLS with 'View As' so important before publishing?

Answer: Because a filter rule that looks correct in theory can still contain mistakes that only become visible once you actually see the restricted data — and a data leak through a broken RLS rule can be a serious real-world problem.

WEEK 4 — Pro Practices • DAY 24 OF 30

Dataflows, Gateways & Incremental Refresh

👨‍🏫 Teacher's Note

Dataflows and incremental refresh are genuinely 'pro' topics that most beginner courses skip entirely, and I keep this lesson intentionally conceptual rather than deeply technical — the goal is recognizing WHY large organizations need these tools, not full operational mastery, which comes with real production experience over time.

What Is a Dataflow?

A **Dataflow** moves the data-cleaning work (the Power Query logic from Week 1) out of individual .pbix files and into a shared, reusable location in the Power BI Service — so multiple reports and multiple report authors can build on the exact same cleaned data, instead of each repeating the same cleanup work separately.

Why Dataflows Matter for Teams

Without a Dataflow	With a Dataflow
Every report author repeats the same Power Query cleaning steps	Cleaning logic is defined ONCE, centrally
A data source change requires updating every report file separately	Update the dataflow once — every connected report benefits
Hard to guarantee consistency across reports	Every report uses the same, guaranteed-consistent cleaned data

🔗 CREATING A DATAFLOW (SERVICE)

1. In a workspace, click **New → Dataflow**
2. Choose a data source, just like Get Data in Desktop
3. Use the same Power Query Editor interface to clean and shape the data
4. Save and refresh the dataflow — it now exists as a reusable source

What Is a Gateway?

An **on-premises data gateway** is a small piece of software installed on a machine inside your organization's network — it lets the Power BI Service securely refresh data from sources that aren't publicly accessible on the internet, like an internal company database.

Without a Gateway	With a Gateway
Power BI Service can't reach an internal-only database	Gateway acts as a secure bridge, relaying the connection
Scheduled refresh fails for on-premises sources	Scheduled refresh works reliably, on a set schedule

What Is Incremental Refresh?

By default, a full data refresh reloads EVERY row every time — fine for small tables, painfully slow for a table with millions of historical rows. **Incremental refresh** configures Power BI to only reload RECENT data, leaving older, unchanged historical data untouched.

🔗 CONFIGURE INCREMENTAL REFRESH (DESKTOP)

1. Create Date range parameters named exactly RangeStart and RangeEnd
2. Filter your main query's date column using these parameters
3. Right-click the table → Incremental refresh
4. Set how much historical data to store, and how much recent data to refresh on each run

Setting example	Effect
Store 5 years, refresh 1 day	Keeps 5 years of history; each refresh only reprocesses the most recent day's data

When These Pro Features Actually Matter

Feature	Reach for it when...
Dataflows	Multiple reports/authors need the exact same cleaned source data
Gateway	Your data lives on an internal server, not a public cloud service
Incremental refresh	A table has millions of rows and full refreshes are too slow

💡 **TIP:** As a beginner, you don't need to implement these features immediately — recognizing what they're FOR and roughly when a growing organization would need them is the valuable takeaway, since hands-on use typically comes with real production experience.

👉 **PRACTICE:** Without necessarily implementing it, write out a short plan: if your Sales Dashboard grew to 10 million historical rows and needed to be refreshed by 5 different report authors, which of today's three features would you introduce, and in what order, and why?

Quick Check

What problem does incremental refresh solve?

Answer: It avoids reloading an entire, very large table on every refresh — instead only reprocessing recent data while leaving older, unchanged historical data untouched, dramatically speeding up refreshes on big datasets.

DAX Variables: Writing Cleaner, Faster Formulas

👨‍🏫 Teacher's Note

DAX variables are one of those features that feels like a minor convenience right up until a student tries to debug a deeply nested formula without them — at which point their value becomes completely obvious. I introduce them by first showing a gnarly, repeated-expression formula, then rewriting it with VAR, and just letting the readability difference speak for itself.

The Problem with Long, Repeated DAX Expressions

DAX

```
Profit Margin % =
DIVIDE(
    SUM(Sales[Revenue]) - SUM(Sales[Cost]),
    SUM(Sales[Revenue])
)
```

Notice SUM(Sales[Revenue]) appears twice — harmless here, but in more complex formulas, repeating the same expression is both hard to read and can hurt performance, since it may be evaluated more than once.

Rewriting with VAR

DAX

```
Profit Margin % =
VAR TotalRevenue = SUM(Sales[Revenue])
VAR TotalCost = SUM(Sales[Cost])
RETURN
    DIVIDE(TotalRevenue - TotalCost, TotalRevenue)
```

RESULT

Region	Profit Margin %
North	38.2%
South	35.7%
<i>Identical result to the version without variables</i>	

VAR names an intermediate calculation once; **RETURN** is where you use those named values in the final expression. This is not just a style preference — it genuinely improves both readability and, in many cases, performance.

Why Variables Help Readability

Without VAR	With VAR
Complex nested expressions repeated multiple times	Each piece calculated once, given a clear name
Hard to tell what a sub-expression represents	TotalRevenue and TotalCost are self-explanatory
Debugging requires mentally re-deriving each piece	You can temporarily RETURN a single variable to inspect it

Debugging with Variables

A genuinely useful trick: temporarily change RETURN to output just one variable, to check its value in isolation while building a complex measure.

DAX

```
Profit Margin % =
VAR TotalRevenue = SUM(Sales[Revenue])
VAR TotalCost = SUM(Sales[Cost])
RETURN
    TotalRevenue -- temporarily checking this one piece is correct first
```

Variables Can Reference Earlier Variables

DAX

```
YoY Growth % =  
VAR CurrentRevenue = SUM(Sales[Revenue])  
VAR PriorRevenue = CALCULATE(SUM(Sales[Revenue]), SAMEPERIODLASTYEAR('CalendarTable'[Date]))  
VAR GrowthAmount = CurrentRevenue - PriorRevenue  
RETURN  
    DIVIDE(GrowthAmount, PriorRevenue)
```

RESULT

Month	YoY Growth %
March 2026	26.5%

Each VAR can build on the ones defined before it, just like a sequence of named steps

A Performance Benefit, Not Just Style

Without variables, DAX may re-evaluate the exact same expression multiple times if it appears more than once in a formula. A variable is calculated ONCE and then reused — on complex formulas over large tables, this can produce a genuinely measurable speed improvement.

 **TIP:** Reach for VAR as soon as any sub-expression appears more than once in a formula, or once a formula has more than 2-3 logical steps — the readability and maintainability payoff is almost always worth the extra few lines.

⇒ **PRACTICE:** Rewrite your Profit Margin % and YoY Growth % measures from earlier days using VAR/RETURN if you haven't already, and add a third measure of your choice that uses at least two variables building on each other.

Quick Check

What are the two genuine benefits of using VAR in a DAX measure, beyond just personal style preference?

Answer: Improved readability (each piece of logic gets a clear, self-explanatory name) and potential performance gains (an expression used in a variable is calculated once, rather than possibly being re-evaluated every time it appears).

WEEK 4 — Pro Practices • DAY 26 OF 30

Iterator Functions: SUMX, AVERAGEX, RANKX

Teacher's Note

Iterator functions (the X-suffixed functions like SUMX) are where DAX asks students to think row by row again, after CALCULATE and context transition trained them to think in aggregates — that back-and-forth is genuinely disorienting the first time. I always connect it back to a spreadsheet metaphor: an iterator is like a formula copied down every row, then summed, all inside one DAX expression.

What Is an Iterator Function?

Regular aggregation functions like SUM() work directly on a whole column. **Iterator functions** (recognizable by their **X** suffix) instead go row by row through a table, calculate something for EACH row, and then combine those row-level results — needed whenever a calculation can't be expressed as a simple column total.

SUMX — Row-by-Row Multiplication, Then Summed

A very common real need: Revenue isn't always a single stored column — sometimes it's Price × Quantity, calculated per row, then totaled.

DAX

```
Total Revenue (calculated) =  
SUMX(  
    Sales,  
    Sales[UnitPrice] * Sales[Quantity]  
)
```

RESULT			
OrderID	UnitPrice	Quantity	Row Revenue (conceptual)
1	60	3	180
2	500	2	1000
SUMX calculates UnitPrice x Quantity for EACH row, then adds all the row results together → 1180			

Why Not Just SUM(UnitPrice * Quantity)?

⚠ **WATCH OUT:** SUM() only works on a single existing column — it cannot perform a row-by-row calculation like multiplying two different columns together. Trying SUM(Sales[UnitPrice] * Sales[Quantity]) directly causes an error; SUMX is specifically built for this row-by-row-then-aggregate pattern.

AVERAGEX — Row-by-Row, Then Averaged

DAX

```
Average Order Value (calculated) =
AVERAGEX (
    Sales,
    Sales[UnitPrice] * Sales[Quantity]
)
```

RESULT	
	Average Order Value
(across all orders)	590

RANKX — Ranking Rows Against Each Other

DAX

```
Customer Revenue Rank =
RANKX (
    ALL (Customers) ,
    [Total Revenue]
)
```

RESULT		
Customer	Total Revenue	Customer Revenue Rank
Aisha Retail Co.	62,000	1
Ravi Traders	28,000	2
Meera Shop	9,500	3
ALL(Customers) ensures ranking happens across ALL customers, not just whatever's currently filtered		

Common Iterator Functions

Function	Row-by-row calculates, then...	Common use
SUMX	...adds all row results together	Revenue = Price x Quantity, totaled
AVERAGEX	...averages all row results	Average order value when it's not a stored column
MAXX / MINX	...finds the largest/smallest row result	Highest single-order revenue
RANKX	...ranks each row against the others	Ranking customers, products, or regions
COUNTX	...counts row results matching a condition	More flexible row-level counting

The Mental Model: Iterators Think Like a Spreadsheet Formula Copied Down

An iterator function behaves conceptually like writing a formula in one Excel cell, copying it down every row of a table, and THEN summarizing that new column — except it all happens inside one DAX expression, without ever creating a real stored column.

🔗 **TIP:** Whenever a calculation needs to multiply, compare, or combine TWO OR MORE COLUMNS on the same row before aggregating, that's the signal you need an iterator (X-suffix) function, not a plain SUM/AVERAGE.

➡ **PRACTICE:** Write a SUMX measure calculating Total Revenue as UnitPrice × Quantity (if your Sales table has these as separate columns), and a RANKX measure ranking products by their Total Revenue.

Quick Check

Why can't you just write SUM(Sales[UnitPrice] * Sales[Quantity]) directly?

Answer: SUM() only operates on a single existing column — it can't perform a row-by-row multiplication between two different columns. SUMX is specifically designed for this row-by-row calculation, then aggregation, pattern.

WEEK 4 — Pro Practices • DAY 27 OF 30

What-If Parameters & Scenario Analysis

👤 Teacher's Note

What-if parameters are one of the most delightful features to teach, because the payoff is immediate and visual — a slider that instantly recalculates an entire report in front of the room. I use this lesson partly as a confidence booster before the more conceptually demanding optimization and design lessons that close out the course.

What Is a What-If Parameter?

A **What-If parameter** creates an interactive slider that lets a report viewer test different assumptions live — like "what if we increased prices by X%?" — and see every connected measure and visual instantly recalculate.

🔧 CREATING A WHAT-IF PARAMETER

1. Modeling tab → New Parameter → Numeric range
2. Set a name (e.g. 'Price Increase %'), minimum, maximum, and increment
3. Power BI automatically creates a new table AND a matching measure for this parameter
4. Add the auto-generated slicer to your report page

Using the Parameter in a Measure

DAX

```
Adjusted Revenue =  
SUM(Sales[Revenue]) * (1 + 'Price Increase %'[Price Increase % Value])
```

RESULT

Price Increase % (slider)	Adjusted Revenue
0%	1,250,000
5%	1,312,500
10%	1,375,000

Moving the slider live-recalculates every visual using this measure

📊 **WHAT YOU'LL SEE:** A slider control on the report canvas labeled 'Price Increase %' — dragging it instantly updates every chart and card built on the Adjusted Revenue measure, with no page reload.

A Practical Scenario: Break-Even Analysis

DAX

```
VAR SelectedDiscount = 'Discount %'[Discount % Value]  
VAR AdjustedRevenue = SUM(Sales[Revenue]) * (1 - SelectedDiscount)  
VAR TotalCost = SUM(Sales[Cost])  
RETURN  
    AdjustedRevenue - TotalCost
```

A viewer can drag a Discount % slider and immediately see at what discount level profit would turn negative — a genuinely powerful, business-relevant "what if" tool built with just a slider and a DAX measure.

Field Parameters — Letting Viewers Choose WHICH Field to Display

A related, newer feature: **Field parameters** let a viewer switch which MEASURE or COLUMN a chart displays entirely, using a

slicer — for example, toggling one chart between showing Revenue, Profit, or Order Count.

🔗 CREATING A FIELD PARAMETER

1. Modeling tab → New Parameter → Fields

2. Add the measures/columns you want the viewer to be able to switch between

3. Add the generated slicer, and use the generated field on your chart's axis or values

📊 WHAT YOU'LL SEE: A small slicer listing 'Revenue', 'Profit', 'Order Count' — clicking a different option instantly changes what the SAME chart displays, without needing three separate charts.

What-If Parameters vs Field Parameters

	What-If Parameter	Field Parameter
Lets viewer change...	A NUMERIC assumption (e.g. a percentage)	WHICH field/measure a visual shows
Typical use	Scenario/sensitivity analysis	One flexible chart instead of several similar ones

💡 TIP: What-if parameters are one of the most genuinely delightful features to demo — the instant, visual feedback of dragging a slider and watching an entire report recalculate tends to be the moment a non-technical audience truly appreciates what Power BI can do.

⇒ **PRACTICE:** Create a What-If parameter for a Discount % (0% to 30%), build an Adjusted Revenue measure using it, and add the generated slicer alongside a chart so dragging it visibly changes the chart in real time.

Quick Check

What does a Field Parameter let a viewer do that a What-If parameter doesn't?

Answer: A Field Parameter lets a viewer switch which MEASURE or COLUMN a visual displays entirely (like toggling between Revenue and Profit on the same chart); a What-If parameter lets them adjust a numeric assumption, like a percentage.

WEEK 4 — Pro Practices • DAY 28 OF 30

Performance Optimization & Best Practices

👨‍🏫 Teacher's Note

Performance optimization is a topic I actively discourage over-applying too early — I tell students plainly: build the correct, clear model and report first, and only optimize once a report actually feels slow to use. Chasing performance on a report that was never actually a problem is one of the most common ways intermediate users waste time for no real benefit.

The Golden Rule of Optimization

Build the correct, clear model and report first. Only optimize once a report actually FEELS slow to use. Chasing performance on a report that was never really a problem wastes time and can make a model harder to maintain for no measurable benefit.

Star Schema — Still the Foundation of Performance

Everything from Day 7's star schema lesson directly affects performance, not just organization. A clean star schema (fact table + simple dimension tables, one-directional 1-to-many relationships) is consistently faster than a large, flat, denormalized table or a tangled web of many-to-many relationships.

Reducing Model Size

Technique	Why it helps
Remove unused columns in Power Query	Every loaded column takes memory, whether it's ever used in a visual or not
Use whole numbers instead of decimals where possible	Whole numbers compress and store more efficiently
Disable auto date/time hierarchies you don't need	Reduces hidden, automatically-generated date tables Power BI creates by default
Avoid importing unnecessary high-cardinality text columns	Columns with many unique text values (like long comments) are expensive to store

🔧 DISABLE AUTO DATE/TIME

1. File → Options and settings → Options → Data Load
2. Uncheck 'Auto date/time for new files'

Writing More Efficient DAX

Prefer	Over	Why
A simple column filter	FILTER() when a simple condition works	Simple filters are generally faster to evaluate
VAR to store repeated sub-expressions	Repeating the same expression multiple times	Avoids potentially re-calculating the same thing twice
Measures over calculated columns	Calculated columns for anything that should respond to filters	Calculated columns bloat model size and don't respond to context

Using Performance Analyzer

🔧 FIND SLOW VISUALS

1. View tab → Performance Analyzer → Start Recording
2. Interact with your report (click slicers, change pages) or click Refresh Visuals
3. Review the recorded times for each visual, sorted from slowest to fastest

OUTPUT

Performance Analyzer results

OUTPUT

Visual 'Sales by Region Chart': DAX query 850ms, Visual display 40ms

This tells you precisely WHERE time is being spent — a slow DAX query points to the measure/model; slow visual display often points to too many data points or overly complex formatting.

Common Performance Mistakes

Mistake	Better approach
One giant table instead of a proper star schema	Split into fact and dimension tables
Too many visuals on one page (15+)	Split across multiple focused pages
Unnecessary bidirectional relationships	Default to single-direction filtering unless truly needed
Complex nested calculated columns	Move logic into measures with VAR where possible

🔗 **TIP:** Performance Analyzer is worth running on any report BEFORE sharing it widely — catching one accidentally slow visual before deployment saves a lot of frustrated messages from real users later.

⇒ **PRACTICE:** Run Performance Analyzer on your Sales Dashboard report while clicking through its slicers, identify the single slowest visual, and investigate whether its DAX or its data volume/formatting is the primary cause.

Quick Check

What's the recommended order of operations when a report feels slow?

Answer: Confirm which specific visual is actually slow using Performance Analyzer FIRST, then investigate and optimize based on evidence — never guess and optimize a visual you merely assume is the problem.

WEEK 4 — Pro Practices • DAY 29 OF 30

Report Design Best Practices

🔑 Teacher's Note

Report design best practices are where I remind students that Power BI is fundamentally a communication tool, not just an analysis tool — the most technically impressive dashboard fails completely if the person it was built for can't quickly find the answer they came for. This lesson intentionally has no new DAX or Power Query content — by Day 29, the tools are there; the judgment is what's left to build.

Power BI Is a Communication Tool First

The most technically correct, DAX-perfect dashboard fails completely if the person it was built for can't quickly find the answer they came for. By Day 29, the tools are all in place — what's left to build is judgment.

Designing for Your Actual Audience

Audience	Design priority
Executives	2-4 KPIs, one page, glanceable in under 10 seconds
Analysts	Detailed tables/matrices, ability to drill down and export
Frontline/operational staff	Simple, focused, task-specific — often just one clear chart or table

The Inverted Pyramid: Most Important Information First

Borrowed from journalism: put the single most important number or insight at the TOP LEFT of a page (where eyes land first), with supporting detail below and to the right.

📊 WHAT YOU'LL SEE: A dashboard layout with 3-4 large Card KPIs across the very top, a primary trend chart directly below them, and supporting detail (tables, secondary charts, slicers) further down or to the side.

A Practical Report Design Checklist

- ☐ Does every page answer ONE clear central question?
- ☐ Are the 2-4 most important numbers visible without any scrolling or clicking?
- ☐ Is a consistent theme, font, and color scheme used across every page?
- ☐ Are titles clear enough that a visual makes sense without additional explanation?
- ☐ Have unnecessary gridlines, borders, and default chart clutter been removed?
- ☐ Does color have a consistent MEANING (e.g. always red = under target), not just decoration?
- ☐ Has this been tested by someone who wasn't involved in building it?

Naming Conventions for a Maintainable Model

Element	Good practice
Measures	Clear business language: 'Total Revenue', not 'Sum_Rev_v2'
Tables	Simple, singular or clearly pluralized: 'Sales', 'Products'
Calculated columns	Rare — and named for what they represent, not how they're built
Report pages	Descriptive tab names: 'Executive Summary', not 'Page 1'

Organizing Measures for a Large Model

In bigger models, create a dedicated, empty "Measures" table to hold all your measures in one organized place, rather than scattering them across whichever data table happened to be selected when each was created.

Documentation Habits Worth Building

- Add a description to key measures (right-click → Properties, or the Description field) explaining what they calculate and why
- Keep a simple README page or document noting data sources, refresh schedule, and who owns the report
- Use clear, consistent Applied Steps names in Power Query (from Day 3) so the data prep logic stays understandable months later

💡 TIP: The single highest-leverage report design habit: before building anything, write down in one sentence the exact question this report page needs to answer for its intended audience. Every design decision afterward should serve that one sentence.

➡ **PRACTICE:** Take your Sales Dashboard report and run it through the full design checklist above. Fix at least 3 issues you find, then have a classmate or colleague view it cold and tell you what they understood within 10 seconds.

Quick Check

What's the 'inverted pyramid' principle in report design?

Answer: Put the single most important information at the top, where a viewer's eyes land first, with supporting detail below — mirroring how a news article leads with the most important fact, not build-up context.

FINAL PROJECT • BEGINNER TO PRO

Project 2 (Capstone): Executive Business Dashboard

Project Overview

Your capstone is a complete **Executive Business Dashboard** — a polished, multi-page Power BI report combining everything from all 30 days: a clean data model, well-written DAX (including time intelligence and iterators), interactive visuals, and professional design.

Requirements

- ☐ A properly modeled star schema (at least one fact table, 3+ dimension tables, including a Date table)
- ☐ At least 8 well-named DAX measures, including at least one using CALCULATE, one time intelligence measure, and one iterator (X-function)
- ☐ At least 3 report pages: an Executive Summary, a Detail/Table page, and a Drill-through page
- ☐ At least one slicer, one bookmark, and consistent theming/formatting across all pages
- ☐ Row-Level Security configured and tested with View As

Suggested Structure

```
Page 1: Executive Summary
- 4 KPI Cards (Total Revenue, Total Profit, YoY Growth %, Unique Customers)
- 1 trend Line chart (Revenue by Month)
- 1 Bar chart (Revenue by Region)
- Region + Date slicers

Page 2: Product Detail (drill-through target from Page 1's category chart)
- Matrix: Category x Sub-Category, values = Revenue, Profit
- Table: full product-level detail

Page 3: Customer Insights
- Treemap: Revenue by Customer Segment
- RANKX-based Top 10 Customers table
- Scatter: Marketing Spend vs Revenue by Customer
```

Step 1: Build the Model

DAX

```
CalendarTable = CALENDAR(MIN(Sales[Date]), MAX(Sales[Date]))
```

🔗 MODEL CHECKLIST

1. Load Sales (fact), Products, Customers, Region, and CalendarTable
2. Build 1-to-many relationships from each dimension table into Sales
3. Mark CalendarTable as the official Date table

Step 2: Core Measures

DAX

```
Total Revenue = SUM(Sales[Revenue])
Total Profit = SUM(Sales[Revenue]) - SUM(Sales[Cost])

Revenue YTD = TOTALYTD([Total Revenue], 'CalendarTable'[Date])
Revenue LY = CALCULATE([Total Revenue], SAMEPERIODLASTYEAR('CalendarTable'[Date]))
YoY Growth % = DIVIDE([Total Revenue] - [Revenue LY], [Revenue LY])

Unique Customers = DISTINCTCOUNT(Sales[CustomerID])

Avg Revenue Per Order =
    DIVIDE(SUMX(Sales, Sales[UnitPrice] * Sales[Quantity]), COUNTROWS(Sales))

Customer Revenue Rank = RANKX(ALL(Customers), [Total Revenue])

North Region Revenue = CALCULATE([Total Revenue], Sales[Region] = "North")
```

Step 3: Build the Report Pages

WHAT YOU'LL SEE: Page 1 shows 4 large Card KPIs across the top, a Line chart of monthly revenue trend below-left, a Bar chart of revenue by region below-right, and Region/Date slicers in a left-hand panel — everything visible without scrolling.

Step 4: Add Row-Level Security

DAX

```
[Region] = "North"
```

🔧 CONFIGURE AND TEST RLS

1. Create a role per region (or a dynamic USERPRINCIPALNAME-based rule)
2. Use View As to confirm each role sees only its intended data
3. Publish, then assign real users to roles in the Service

Self-Assessment Rubric (for teaching use)

Criteria	Points
Star schema is correctly modeled with a proper Date table	20
8+ measures including CALCULATE, time intelligence, and an iterator	25
3+ pages including a working drill-through	20
Formatting, theme, and layout are clean and consistent	15
Row-Level Security is configured and verified with View As	10
Bonus: What-if parameter or field parameter included	10

Extend the Project (Recommended)

- ☐ Add a What-If parameter for a sales target and a Gauge visual tracking progress toward it
- ☐ Publish to the Service and share it with a classmate, then have them view it as a restricted RLS role
- ☐ Run Performance Analyzer and optimize the single slowest visual
- ☐ Add bookmarks for a guided "walkthrough" sequence through the Executive Summary page

TIP: This capstone is intentionally open-ended past the core requirements. If you're teaching this course, treat the extensions as the real differentiator between students — anyone can follow the steps above, but choosing and correctly implementing a good extension shows real dashboard design judgment.

30-Day Journey Recap

Week	Focus
Week 1	Power Query, data cleaning, data types, and star schema modeling
Week 2	DAX fundamentals: measures, CALCULATE, time intelligence, filters — and Project 1: Sales Dashboard Model
Week 3	Visuals, interactivity, formatting, drill-through, tables and matrices
Week 4	Publishing, RLS, advanced DAX, optimization, design — and Project 2: Executive Dashboard

Where to Go Next

- Explore **DAX Studio** and **Tabular Editor**, free external tools professionals use for advanced model work
- Study more advanced DAX patterns: many-to-many bridge tables, budget-vs-actual models
- Practice on real public datasets to build a portfolio of dashboards
- Learn Power BI's integration with Power Automate and Power Apps for the broader Power Platform
- Teach someone else — explaining a concept is the fastest way to master it

⇒ **PRACTICE:** Build the complete Executive Business Dashboard exactly as outlined, verify every requirement in the checklist above, then implement at least one extension from the list on your own.